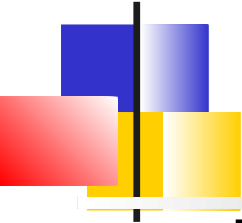
A decorative graphic on the left side of the slide consists of a vertical black line. To its left are three overlapping squares: a blue one at the top, a red one in the middle, and a yellow one at the bottom. To the right of the line are two yellow squares, one above the other.

Jenkins Pipelines

David THIBAU – 2026

david.thibau@gmail.com



Agenda

Démarrage

- Plateforme de CI/CD
- Le projet Jenkins
- Installation
- Interface utilisateur

Concepts

- Configuration Système, Outils et plugins
- Noeuds et Clouds
- Exécution des Jobs

Pipelines

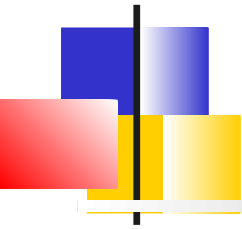
- Approche et concepts
- Syntaxe déclarative
- Groovy et syntaxe script
- Bibliothèques partagées

Container

- Docker
- Kubernetes

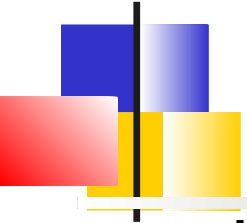
Annexes

- Tests et analyse statique
- Infrastructure as Code
- Le plugin JTE
- Syntaxe script



Démarrage

Pipeline CI/CD
Installation
Interface Utilisateur

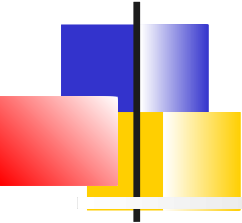


Plateforme d'intégration continue

L'intégration continue dans sa forme la plus simple consiste en un outil surveillant les changements dans le **Source Control Management (SCM)**

Lorsqu'un changement est détecté, l'outil construit, teste et éventuellement déploie automatiquement l'application

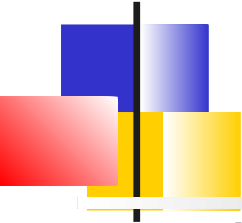
Si ce traitement échoue, l'outil notifie immédiatement les développeurs afin qu'ils **corrigent le problème ASAP**



En continu

A chaque ajout de valeur dans le dépôt de source (push), l'intégralité des tâches nécessaires à la mise en service d'un logiciel (intégration, tests, déploiement) sont essayées.

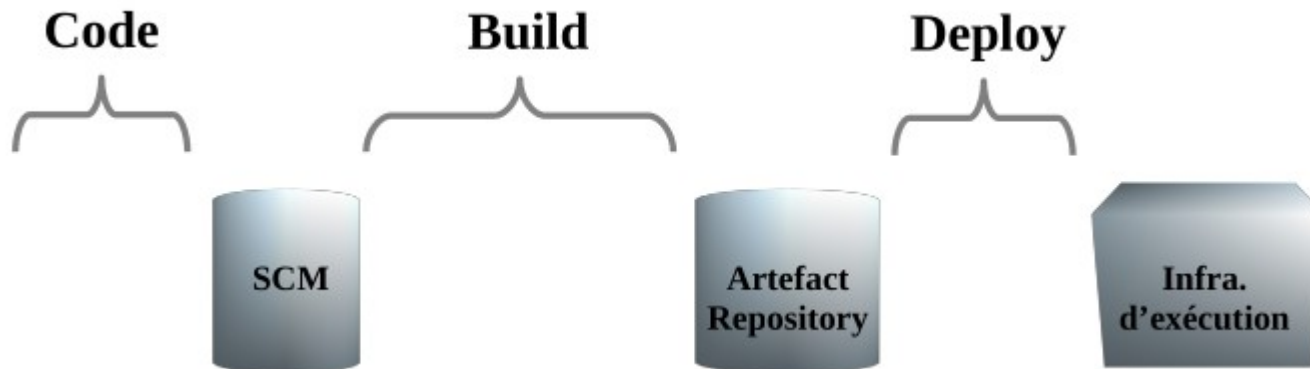
En fonction de leurs succès, l'application est déployée dans les différents environnements : intégration, staging, production

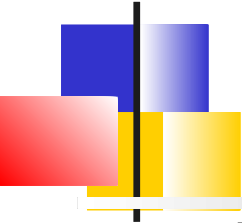


Build / Release / Run

La pipeline suit le cycle de vie du code :

- 1) Le code est testé localement puis poussé dans le **dépôt de source**.
- 2) Le build construit l'artefact et si le build est concluant stocke une release dans un **dépôt d'artefact**.
- 3) L'outil de déploiement accède aux différentes release et les déploie sur **l'infra d'exécution**

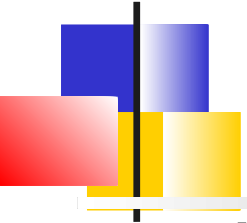




Build is tests !

La construction de l'application consiste principalement à :

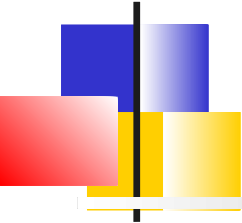
- Packager le code source dans un format exécutable, déployable
- Effectuer toutes les vérifications automatiques permettant d'avoir confiance dans l'artefact généré et autoriser son déploiement.
(Tests Unitaires/Intégration, Fonctionnel, Performance, Sécurité, Licenses, ...)



Pipelines

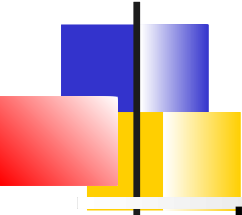
Les étapes de construction automatisées sont séquencées dans une **pipeline**.

- Une étape est exécutée seulement si les étapes précédentes ont réussi.
- Les plate-formes d'intégration/déploiement continu ont pour rôle de démarrer et observer l'exécution de ces pipelines



Distinction CI/CD





Outil de communication

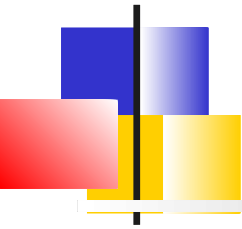
La plateforme a pour vocation de publier les résultats des builds :

- Nombre de tests exécutés, succès/échecs
- Couverture des tests
- Complexité, Vulnérabilités du code source
- Performance : Temps de réponse/débit
- Documentation, Release Notes
- ...

Les métriques sont visibles de tous en toute transparence

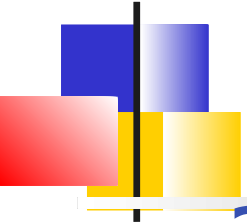
=> Confiance dans ce qui a été produit

=> Motivation pour s'améliorer



Démarrage

Pipeline CI/CD
Installation
Interface Utilisateur



Historique

2004 : Démarrage du projet par Kohsuke Kawaguchi au sein de Sun.

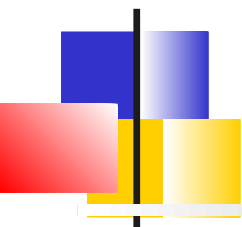
2010 : 70 % du marché

2011 : Rachat de Sun par Oracle et divergences entre l'équipe initiale de développement et Oracle
=> Fork du projet Hudson et création de Jenkins qui reste dans le mode OpenSource

2014 : La société CloudBees emploie la plupart des committers Jenkins et supporte commercialement la solution

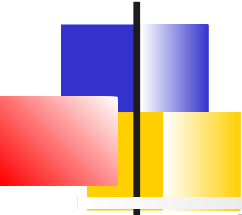
2016 : Pipeline As Code

Aujourd'hui : Toujours très déployé, aux côtés de GitHub Actions et GitLab CI



Atouts

- Facilité d'installation
- Interface web intuitive avec aide intégrée
- Très extensible et adaptable à des besoins spécifiques (Nombreux plugins opensource)
- Communauté large
- Release quasi-hebdomadaires ou LTS release (Long Term Support)
- S'appuie sur un langage de scripting pour exécuter les pipelines

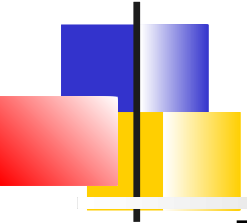


Exécution

Jenkins est un programme exécutable Java qui intègre un serveur Web intégré

Les distributions typiques sont :

- Packages natif Linux/Mac Os
- Service Windows
- Image Docker
- Helm Kubernetes
- Application Java Standalone



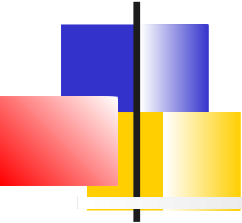
Installation manuelle

Récupérer la distribution et la placer dans le répertoire de votre choix :

- Linux : /usr/local/jenkins ou /opt/jenkins
- Windows : C:\Outils\Jenkins

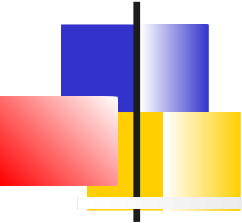
Pour démarrer, exécuter :

```
$ java -jar jenkins.war
```



Options du script de démarrage

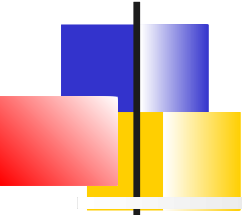
- httpPort** : Port http (Par défaut 8080)
- httpsPort** : Port https (--ajp13Port a été supprimée)
- controlPort** : Démarrage/arrêt du serveur
- prefix** : Chemin de contexte pour l'application web.
- daemon** : Si Unix possibilité de démarrer Jenkins comme daemon.
- logfile** : Emplacement du fichier de log de Jenkins (par défaut le répertoire courant)



Étapes post-installation

Quelques étapes terminent l'installation :

- Déverrouillage de Jenkins (via un mot de passe généré)
- Création d'un administrateur
- Installation de plugins. L'assistant propose d'installer les plugins les plus répandus.

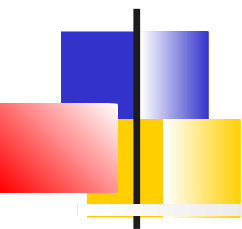


JENKINS_HOME

Au démarrage de l'application web, Jenkins recherche son répertoire JENKINS_HOME dans cet ordre :

- 1) Une entrée dans l'environnement JNDI nommée JENKINS_HOME
- 2) Une propriété système nommée JENKINS_HOME
- 3) Une variable d'environnement nommée JENKINS_HOME
- 4) Le répertoire .jenkins dans le répertoire de l'utilisateur

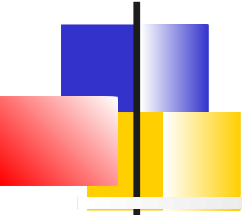
L'intégralité de l'état du serveur est stocké sous le répertoire JENKINS_HOME



Structure de répertoires

Sous JENKINS_HOME, on trouve :

- **jobs** : Configuration des jobs gérés par Jenkins ainsi que les artefacts générés par les builds
- **workspace** : Les workspaces des builds exécutés par le contrôleur
- **nodes** : Configuration des agents
- **plugins** : Les plugins installés .
- **tools** : Les outils installés
- **secrets** : Mots de passes, créidentiels, token
- **fingerprints** : Traces des empreintes des artefacts générés lors des build.
- **updates** : Répertoire interne à Jenkins stockant les plugins disponibles
- **userContent** : Répertoire pour déposer son propre contenu (<http://myserver/userContent>).
- **users** : Les utilisateurs Jenkins si l'annuaire Jenkins interne est utilisé
- **war** : L'application web Jenkins décompressée



Où Jenkins stocke quoi

Trois choses distinctes, stockées à trois emplacements :

Configuration du job : `$JENKINS_HOME/jobs/<job>/config.xml`

- Un répertoire par job ; dossiers et pipelines multi-branches imbriquent leur propre jobs/

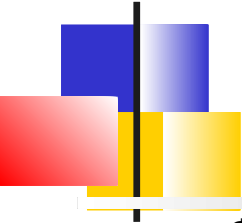
Workspace : sources et fichiers produits par le build

- Sur le contrôleur : `$JENKINS_HOME/workspace/<job>`
- Sur un agent : `<racine de l'agent>/workspace/<job>`
- Jetable : recréé au build suivant, ne pas y stocker d'état

Historique des builds :

`$JENKINS_HOME/jobs/<job>/builds/<n>/`

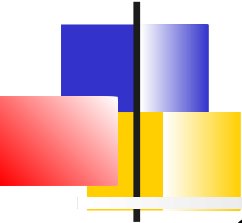
- Conservé selon la politique de rétention du job



Arborescence d'un job

\$JENKINS_HOME/

- |— config.xml *configuration globale*
- |— jobs/
 - |— mon-appli/
 - |— config.xml *définition du job*
 - |— nextBuildNumber
 - |— builds/ *historique*
- |— workspace/
 - |— mon-appli/ *sources + fichiers produits*
- |— nodes/ *configuration des agents*
- |— plugins/
- |— secrets/
- |— users/
- |— fingerprints/

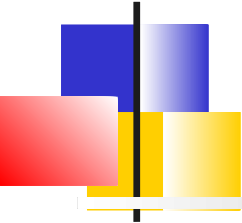


Répertoire d'un build

Chaque build est stocké dans un répertoire numéroté :
\$JENKINS_HOME/jobs/<job>/builds/<n>/

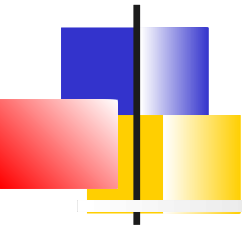
On y trouve :

- **build.xml** : résultat, durée, cause du déclenchement, paramètres
- **log** : la sortie console
- **changelog.xml** : les commits depuis le build précédent
- **archive/** : les artefacts archivés par archiveArtifacts
- Les données publiées par les post-actions (junitResult.xml, ...)
- **workflow/** : l'état d'exécution de la pipeline (jobs Pipeline)



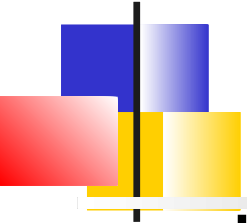
Exemple : répertoire builds

```
jobs/mon-appli/builds/
├─ 1/
├─ 2/
├─ 3/
│   ├─ build.xml
│   ├─ log
│   ├─ changelog.xml
│   ├─ junitResult.xml
│   ├─ archive/
│   │   └─ target/
│   │       └─ mon-appli-1.0.0.jar
│   └─ workflow/ jobs Pipeline
├─ legacyIds
└─ permalinks lastSuccessfulBuild, ...
```



Démarrage

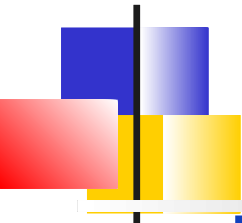
Pipeline CI/CD
Installation
Interface Utilisateur



Concepts de base

L'interface utilisateur de Jenkins propose :

- Validation à la volée des champs de formulaire
- Rafraîchissement automatique
- Aide contextuelle
- Internationalisation
- Liens permanents
- API REST



Chemin de fer

Page d'accueil de type Tableau de bord qui donne l'état de santé des différents projets/builds

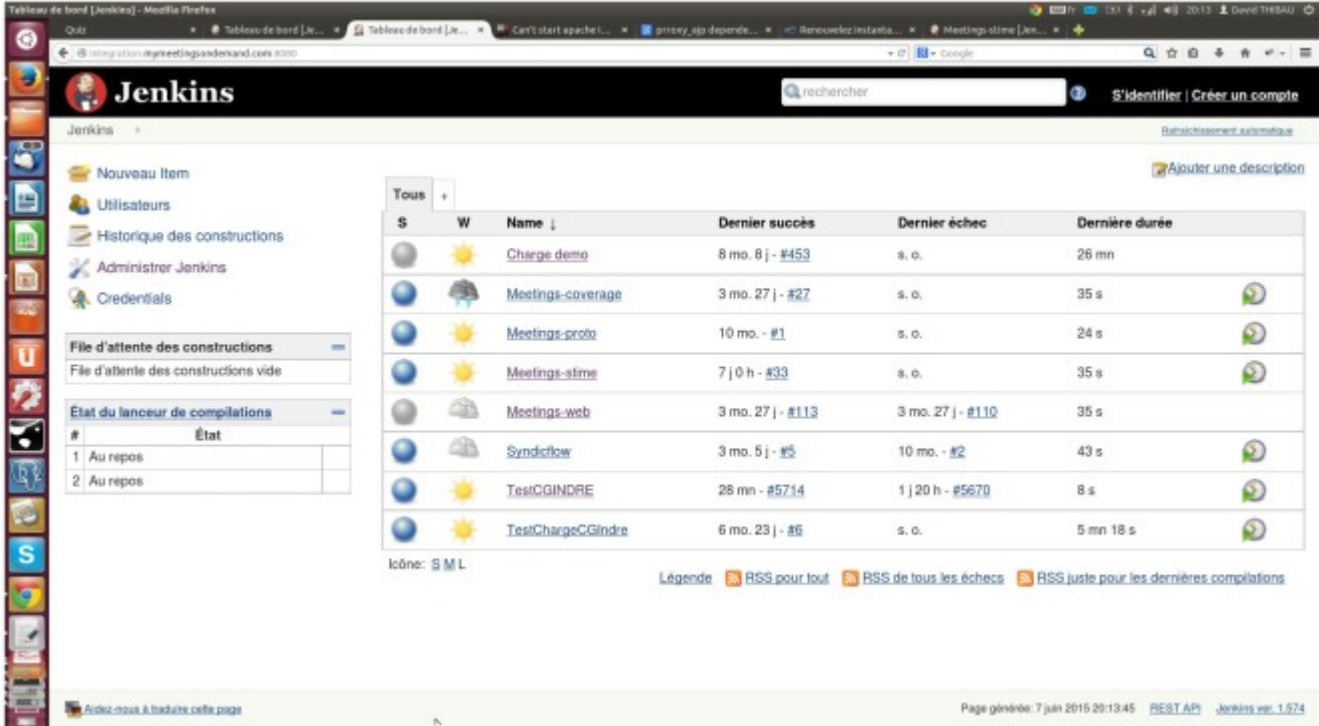
Page projet : Liste les jobs effectués, affiche des graphiques de tendance

Page job : Accès aux traces de la console, à la cause de démarrage aux artefacts créés

L'administrateur a également accès à toute la partie admin, nœuds, plugins, outils, monitoring, ...

L'interface est personnalisable en fonction des utilisateurs

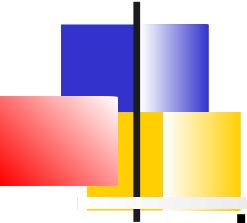
Tableau de bord



The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo, a search bar, and links for 'S'identifier' and 'Créer un compte'. The left sidebar contains various management links like 'Nouveau Item', 'Utilisateurs', and 'Historique des constructions'. The main content area displays a table of build jobs.

S	W	Name ↓	Dernier succès	Dernier échec	Dernière durée
●	☀	Charge demo	8 mo. 8 j - #453	s. o.	26 mn
●	☁	Meetings-coverage	3 mo. 27 j - #27	s. o.	35 s
●	☀	Meetings-prod	10 mo. - #1	s. o.	24 s
●	☀	Meetings-stime	7 j 0 h - #33	s. o.	35 s
●	☁	Meetings-web	3 mo. 27 j - #113	3 mo. 27 j - #110	35 s
●	☁	Syndicflow	3 mo. 5 j - #5	10 mo. - #2	43 s
●	☀	TestCGINDRE	28 mn - #5714	1 j 20 h - #5670	8 s
●	☀	TestChargeCGIndre	6 mo. 23 j - #6	s. o.	5 mn 18 s

Below the table, there is a legend for the status icons: 'Légende' followed by 'RSS pour tout', 'RSS de tous les échecs', and 'RSS juste pour les dernières compilations'. The footer of the page indicates the page was generated on 7 juin 2015 at 20:13:45, with links for 'REST API' and 'Jenkins ver. 1.574'.

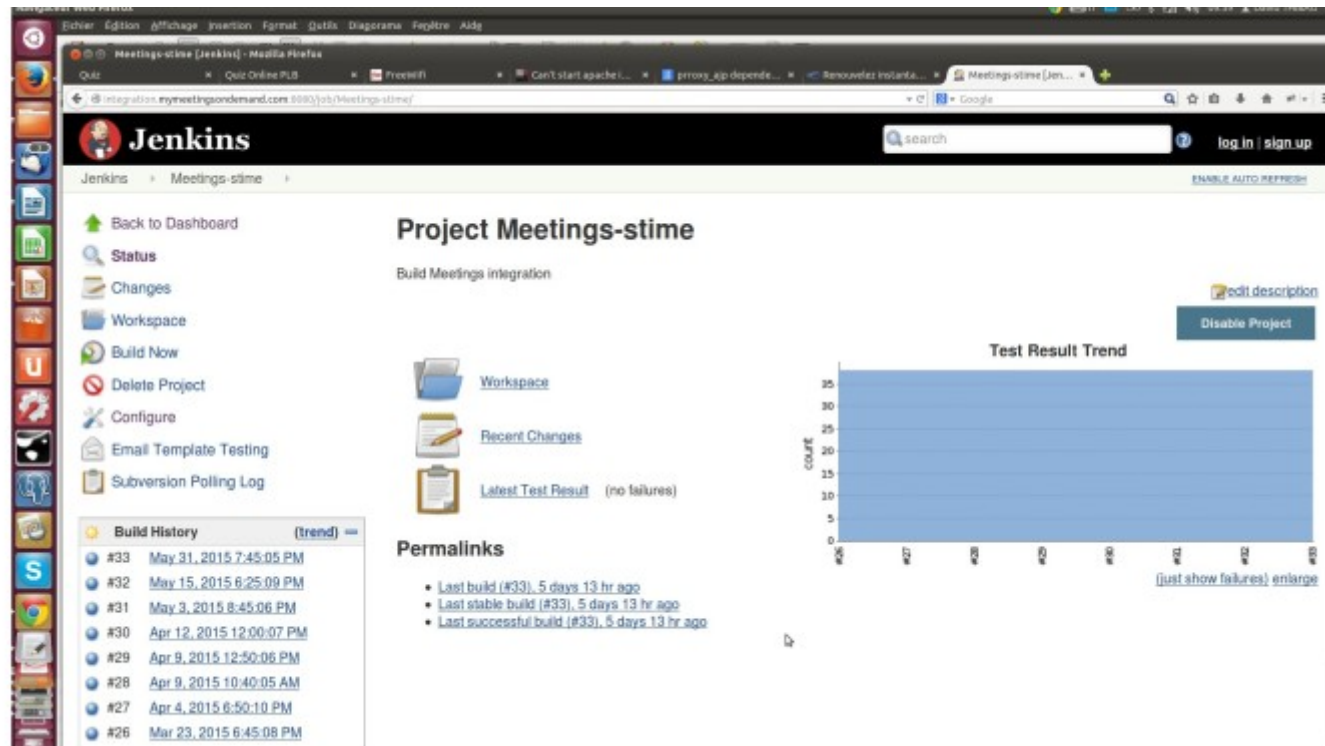


Page projet

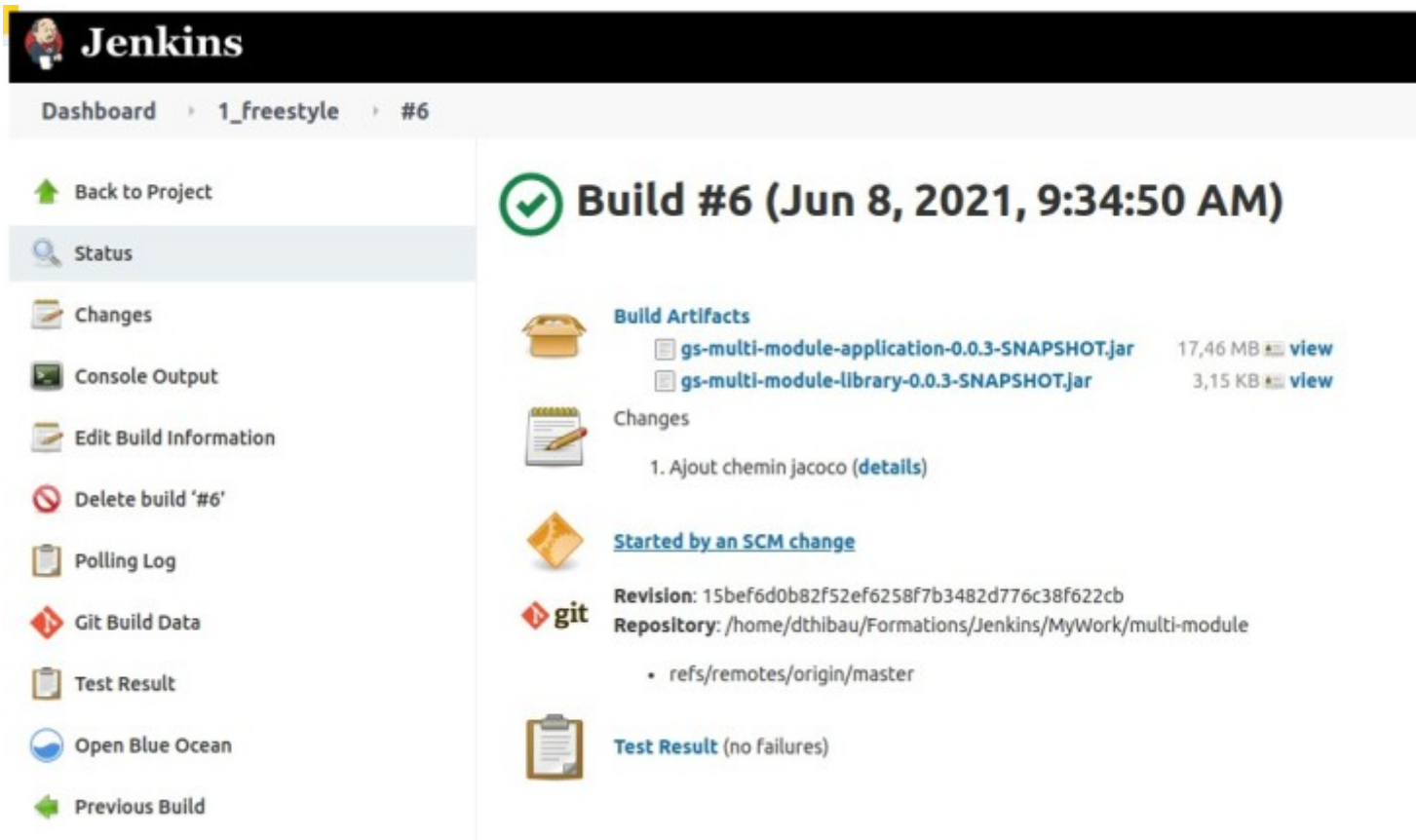
La page projet permet :

- De configurer le projet
- Visualiser les graphiques de tendances (test, temps de build, métriques, ...)
- Accéder à l'espace de travail
- Démarrer manuellement un build
- Voir les changements récents (commit)
- Accéder aux derniers builds (liens permanents)
- Accéder aux builds liés (amont ou aval)

Page projet



Page Build



The screenshot shows the Jenkins web interface for a specific build. The top navigation bar is black with the Jenkins logo and name. Below it, a breadcrumb trail shows 'Dashboard' > '1_freestyle' > '#6'. The left sidebar contains a list of links: 'Back to Project', 'Status' (highlighted), 'Changes', 'Console Output', 'Edit Build Information', 'Delete build '#6'', 'Polling Log', 'Git Build Data', 'Test Result', 'Open Blue Ocean', and 'Previous Build'. The main content area displays 'Build #6 (Jun 8, 2021, 9:34:50 AM)' with a green checkmark icon. Below this, there are sections for 'Build Artifacts' (listing two JAR files with sizes and 'view' links), 'Changes' (listing one change with a 'details' link), 'Started by an SCM change', 'Revision' (15bef6d0b82f52ef6258f7b3482d776c38f622cb), 'Repository' (/home/dthibau/Formations/Jenkins/MyWork/multi-module), and 'Test Result' (no failures).

Jenkins

Dashboard > 1_freestyle > #6

[Back to Project](#)

[Status](#)

[Changes](#)

[Console Output](#)

[Edit Build Information](#)

[Delete build '#6'](#)

[Polling Log](#)

[Git Build Data](#)

[Test Result](#)

[Open Blue Ocean](#)

[Previous Build](#)

Build #6 (Jun 8, 2021, 9:34:50 AM)

Build Artifacts

Artifact Name	Size	Action
gs-multi-module-application-0.0.3-SNAPSHOT.jar	17,46 MB	view
gs-multi-module-library-0.0.3-SNAPSHOT.jar	3,15 KB	view

Changes

1. Ajout chemin jacoco ([details](#))

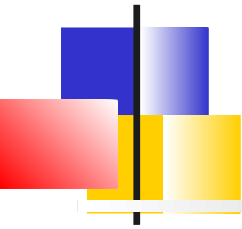
Started by an SCM change

Revision: 15bef6d0b82f52ef6258f7b3482d776c38f622cb

Repository: /home/dthibau/Formations/Jenkins/MyWork/multi-module

- refs/remotes/origin/master

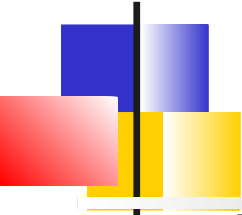
Test Result (no failures)



Concepts

Configuration Système, Outils, Plugins

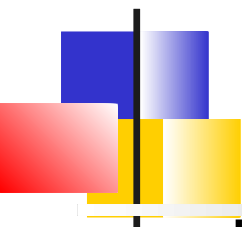
Nœuds et Clouds
Exécution des Jobs



Administration

La page d'administration regroupent des liens par catégorie

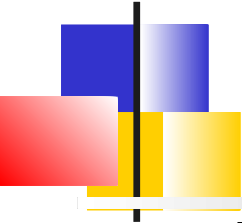
- Configuration système : Intégration, Outils, Plugins, Noeuds et clouds
- Sécurité : Stratégie de sécurité, stockage de créidentiels, utilisateurs, rôles
- Monitoring : Statut, traces du serveur. Analyse de la charge
- – Autres: Redémarrage, exécution de script



Introduction

La configuration du système englobe :

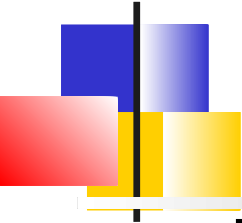
- Configurer le système : Fonctionnement global, configuration du contrôleur, mail de l'administrateur, déclaration de serveur tiers ...
- Configuration des outils : Définition de chemins d'accès à des outils utilisés par les builds : JDK, Maven, SonarScanner, ...
- Gestion des plugins : Disponibilité, installation de plugin
- Gérer les nœuds et les clouds: Ajout/suppression d'agents de build
- Configuration As Code: Gérer la configuration de Jenkins via des fichiers



Configuration système

La constitution de la page dépend des plugins installés, Citons :

- L'emplacement Jenkins : URL et mail de l'administrateur
- Adresse du serveur de mail pour notifier les utilisateurs (Plugin Mailer)
- Emplacement du dépôt local Maven (Plugin Maven Integration)
-



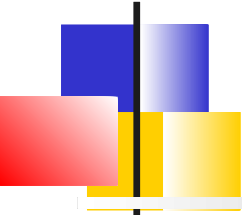
Configuration notification email

La technique principale de notification utilisée par Jenkins se base sur les emails.

Typiquement, Jenkins envoie un email au développeur ayant committé les changements qui ont provoqué l'échec du build

E-mail Notification

SMTP server	smtp.plbformation.com
Default user e-mail suffix	
☒ Use SMTP Authentication	
User Name	stageojen@plbformation.com
Password	*****
Use SSL	☐
SMTP Port	
Reply-To Address	



Configuration globale des outils

Certains outils utilisés lors des builds peuvent être configurés dans cette page.

- Si l'outil est pré-installé sur la machine exécutant le build, il faut spécifier l'emplacement du répertoire HOME
- Sinon, il faut demander à Jenkins de l'installer automatiquement (téléchargement dans le répertoire `$JENKINS_HOME/tools`)

Différentes versions d'un même outil peuvent être configurées

Configuration outils : JDKs

JDK

JDK installations

Add JDK

JDK
Name

JAVA_HOME

☐ Install automatically [?](#)

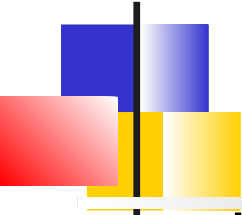
Delete JDK

JDK
Name

JAVA_HOME

☐ Install automatically [?](#)

Save **Apply**



Gestion des plugins

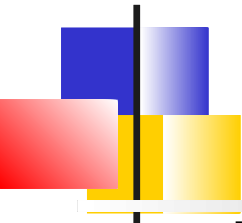
Une page spécifique est dédiée à la gestion des plugins.

L'instance du serveur se connecte au dépôt `updates.jenkins.io`

La page permet de voir :

- Les plugins installés
- Les plugins disponibles
- Les mises à jour disponibles

L'installation d'un plugin ne nécessite généralement pas de redémarrage. Des dépendances existent entre les plugins



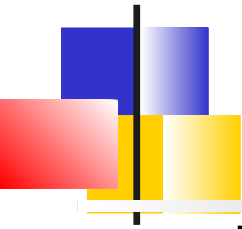
Gestion des créden*t*iels

Jenkins doit accéder à de nombreux créden*t*iels afin de s'authentifier sur les outils associés (SCM, ssh, Serveurs LDAP, Nexus, ...)

Un administrateur système peut configurer des créden*t*iels pour une utilisation dédiée par Jenkins

- Des scopes sont associés aux créden*t*iels et limitent ainsi leur utilisation
- Les jobs et les pipelines peuvent ensuite avoir accès à ces créden*t*iels via des Ids

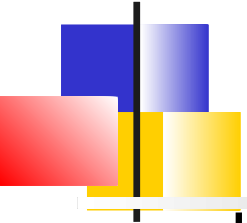
Ces fonctionnalités sont apportées par le plugin
Credentials Binding Plugin



Scopes

Les crédentels stockés par Jenkins peuvent être utilisés :

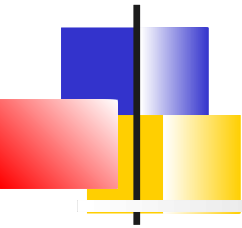
- Globalement (partout dans Jenkins)
- Par un projet spécifique (et ses sous projets pour un projet Dossier par exemple)
- Par un utilisateur Jenkins particulier



Types de crédits

Jenkins stocke des crédits de différents types :

- **Texte secret** : comme un token par exemple (exemple Token GitHub),
- **Username et password** : Traité comme des composants séparé ou comme une chaîne séparé par un :
- **Secret file** : Une chaîne secrète stockée dans un fichier
- Un utilisateur SSH avec sa clé privé
- Un **certificat PKCS#12** et un mot de passe optionnel
- **Certificat d'un hôte Docker**

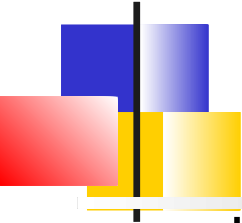


Concepts

Configuration Système, Outils, Plugins

Nœuds et Clouds

Exécution des Jobs



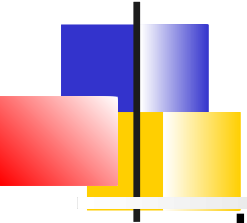
Introduction

Une des fonctionnalités les plus puissante de Jenkins est sa capacité à distribuer les jobs sur des machines distantes

- On peut mettre en place une ferme de serveurs (agents) afin de répartir la charge ou d'exécuter les jobs dans différents environnements

Jenkins a suivi les évolutions des technologies de provisionnement d'infrastructure et propose donc différentes alternatives :

- Serveur matériels ou VM pré-provisionnés
- VMs associés à des outils de gestion de conf
- Clouds privés ou publics utilisant la containerisation

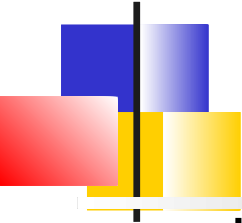


Provisionnement

Le provisionnement des agents consiste à configurer une machine avec les outils de build, les comptes user et les services demandés par un projet.

Différentes alternatives pour le provisionnement sont possibles :

- Provisionnement manuel ou par outils de gestion de conf
- Installation automatique d'outil
- Utilisation dynamique d'image définie dans la pipeline

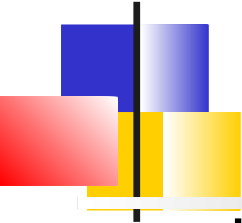


Architecture Jenkins

Jenkins utilise une architecture contrôleur/agent (anciennement « maître/esclave »)

- Le nœud contrôleur
 - gère le démarrage des jobs, les distribue sur les agents, surveille les agents, enregistre et présente les résultats des builds.
 - Il peut éventuellement exécuter lui même des jobs.
- Les nœuds agents exécutent les jobs qu'on leur a demandé.

Il est possible de configurer un projet afin qu'il s'exécute sur certains agents ou de laisser Jenkins choisir un agent.



Nœud agent

Un agent est un petit exécutable Java s'exécutant sur une machine distante et écoutant les requêtes provenant du contrôleur.

- Les agents peuvent s'exécuter sur différents systèmes d'exploitation, dans un container
- Ils peuvent avoir différents outils pré-installés
- Ils peuvent être démarrés de différentes façons selon le système d'exploitation et l'architecture réseau
- Ils proposent un certain nombre d'exécuteurs

Une fois démarrés, ils communiquent avec le contrôleur via TCP/IP ou WebSocket

Netflix 2012 (historique)

1 contrôleur avec 700 utilisateurs

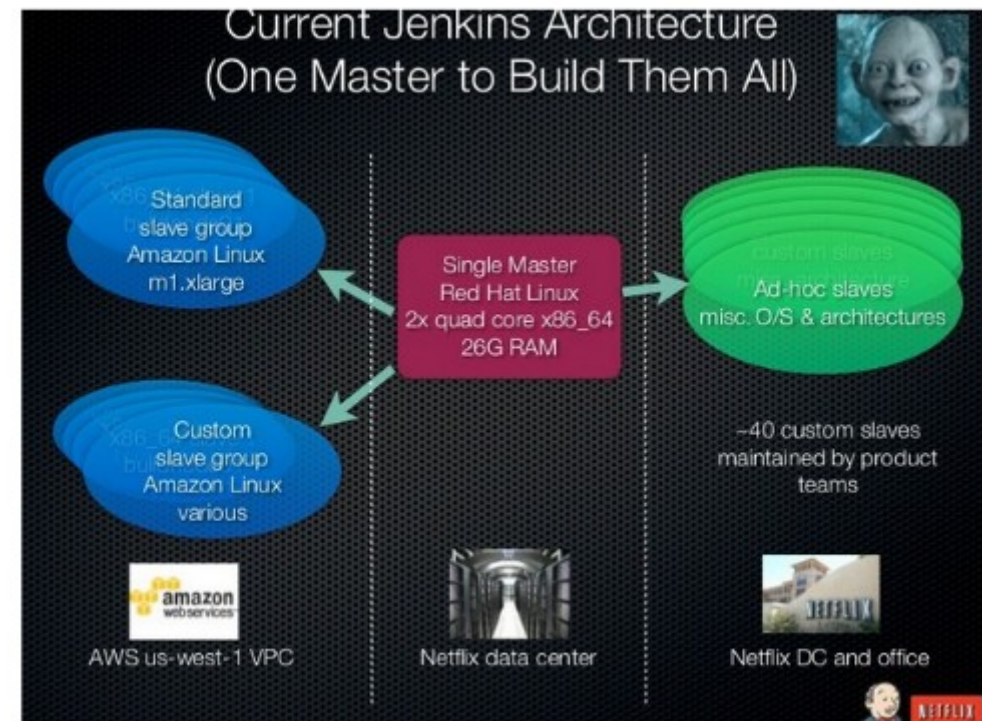
1,600 jobs :

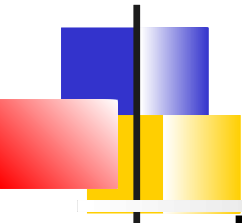
- 2,000 builds/jour
- 2 TB
- 15% build failures

=> 1 contrôleur avec 26Go de RAM

=> Agent Linux sur amazon

=> Agents Mac. Dans le réseau interne





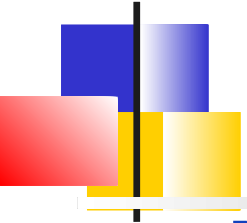
Type de nœud

Par défaut (sans installation de plugins supplémentaire), les agents sont de type « Agent permanent »

- Cela signifie que ce sont des exécutables toujours démarrés (serveur matériel ou VM)

D'autres plugins ajoutent des types de nœuds permettant par exemple le provisionnement dynamique d'agents

Dans les dernières versions, ces nœuds sont gérés via le menu Cloud



Champs d'un nœud

Nom : Identifiant unique

Description : Aide pour les administrateurs

Nombre d'exécuteurs : Nombre de jobs en //

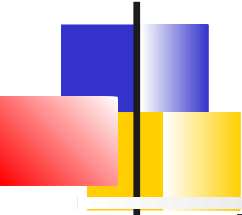
Home : Répertoire Home de travail (ne nécessite pas de backup, chemin spécifique à l'OS)

Labels ou tags : Permet de sélectionner des nœuds

Usage : Autant que possible ou dédié à un job particulier

Méthode de démarrage : agent entrant (JNLP/WebSocket), SSH, Service Windows

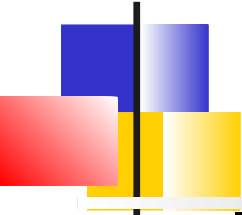
Disponibilité : Le nœud peut être mis offline et démarré seulement lorsque la charge est importante



Démarrage des agents permanents

Différentes alternatives sont possibles pour démarrer les agents permanents

- Le contrôleur démarre les agents via ssh (le plus commun dans un environnement Unix/Linux, avec option pour non-blocking I/O)
- L'agent est démarré en mode entrant (inbound) via JNLP ou WebSocket – Java Web Start a été supprimé
- L'agent est installé comme service Windows

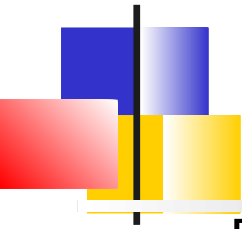


Démarrage via ssh

Il est alors nécessaire :

- d'installer le plugin SSH Build Agents (ex-« SSH Slaves »). Le plugin ajoute un nouveau choix dans le champ « démarrage » lors de la configuration du nœud.
- de fournir les informations de connexion aux agents (hôte, créidentiels)
- Ou installer la clé publique ssh du contrôleur dans `~/.ssh/authorized_keys` de l'agent

Jenkins fait le reste : copie du binaire nécessaire, démarrage et arrêt des agents



Labels/Tags des nœuds

Des labels peuvent être associés à des agents afin de les différencier

Ils peuvent être utilisés :

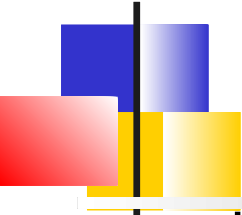
- Pour indiquer qu'un nœud a certains outils installés
- Qu'il s'exécute sur tel OS
- Sa situation géographique ou réseau

Exemple :

`jdk windows eu-central docker`

On peut alors fixer des contraintes concernant le nœud pour un job particulier.

- Contrainte simple : `eu-central`
- Contrainte multiple : `docker && eu-central`



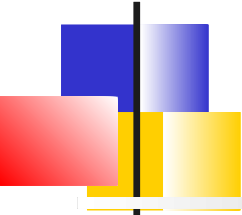
Surveillance des agents

Jenkins surveille également les agents, il peut déclarer un nœud offline si il considère qu'il est incapable d'exécuter un build

3 métriques sont surveillées

- Le temps de réponse : un temps de réponse bas peut indiquer un problème réseau ou que la machine agent est à l'arrêt
- Les ressources disque : l'espace disque, l'espace pris par les répertoires temporaires et l'espace de swap disponible
- Les horloges : elles doivent rester synchronisées avec l'horloge du contrôleur

=> Si un de ces critères n'est pas correct, Jenkins déclare le nœud offline



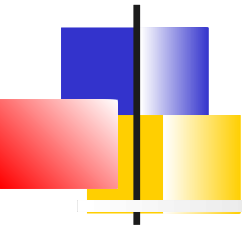
Agents dans le cloud

Jenkins peut provisionner dynamiquement des agents sur le cloud :

- Les agents sont éphémères et existent uniquement pendant l'exécution des jobs.
- Le contrôleur Jenkins s'y connecte via SSH, JNLP ou WebSocket selon la configuration.

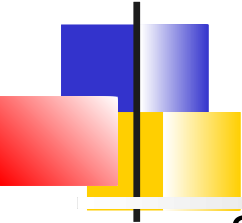
Plugins courants :

- Amazon EC2 (AWS)
- Kubernetes plugin (le plus utilisé aujourd'hui)
- OpenStack plugin
- (Historiquement) JClouds pour divers fournisseurs



Concepts

Configuration Système, Outils, Plugins
Nœuds et Clouds
Exécution des Jobs



Job Freestyle

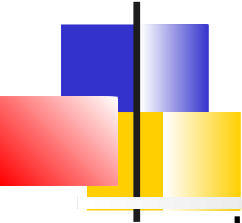
Sans plugin installé, Jenkins propose un seul type de job :

Job FreeStyle : Script shell ou .bat Windows

La configuration d'un job consiste en :

- Des **configurations générales** : Nom, conservation des vieux builds, ...
- L'association à un **SCM**
- La définition des **déclencheurs** de build
- Les **étapes du build** et les **étapes après le build** (choix dépendant des plugins installés)

Le nom du projet est utilisé comme répertoire et dans des URLs => éviter les espaces et les accents



SCM et déclencheurs

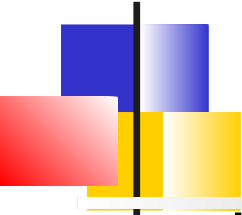
La plupart des jobs sont reliés à un SCM : le démarrage effectue un check-out complet du projet dans l'espace de travail, puis lance le build

Il y a 4 façons de déclencher un job :

- A la fin d'un autre build
- Périodiquement, syntaxe CRON
- En surveillant le SCM, et en déclenchant le build si un changement est détecté
- Manuellement

On peut également démarrer un build via l'API Jenkins (REST ou CLI) :

`http://SERVER/job/PROJECTNAME/build?token=SECRET`



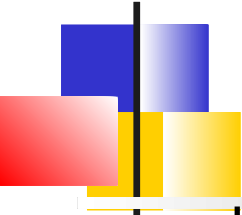
Étapes de build

Un job freestyle est organisé en étapes ; l'échec de l'une fait échouer le build

Les étapes proposées par l'UI dépendent des plugins installés (par défaut : Maven, Bat Windows, Shell)

Exécuter un shell ou un script (typiquement stocké dans le SCM) :

- Le script est indiqué relativement à la racine du répertoire de travail
- Les scripts Shell sont exécutés avec l'option -ex. La sortie apparaît sur la console
- Si une commande retourne une valeur $\neq 0$, le build échoue



Variables d'environnement Jenkins (1)

Jenkins positionne des variables d'environnements qui peuvent être utilisées dans les jobs :

BUILD_NUMBER : N° de build.

BUILD_ID : Un timestamp de la forme YYYY-MM-DD_hh-mm-ss.

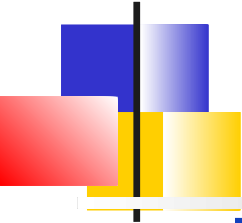
JOB_NAME : Le nom du job

BUILD_TAG : Identifiant du job de la forme jenkins-\${JOB_NAME}-\${BUILD_NUMBER}

EXECUTOR_NUMBER : Un identifiant de l'exécuteur

NODE_NAME : Le nom de l'agent exécutant le build ou "built-in" si le contrôleur

NODE_LABELS : La liste des libellés associés au nœud exécutant le build



Variables d'environnement Jenkins (2)

JAVA_HOME : Le home du JDK utilisé

WORKSPACE : Le chemin absolu de l'espace de travail

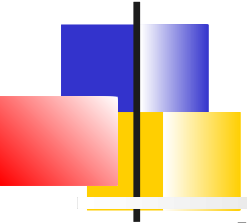
JENKINS_URL : L'URL du serveur Jenkins

JOB_URL : L'URL du job, par exemple :
`http://ci.acme.com:8080/jenkins/game-of-life.`

BUILD_URL : L'URL du build, par exemple
`http://ci.acme.com:8080/jenkins/game-of-life/20.`

SVN_REVISION : La version courante SVN si applicable.

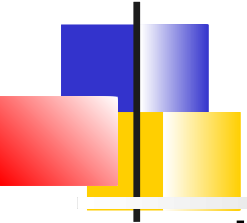
GIT_COMMIT : Identifiant du commit Git



Actions « Post-build »

Lorsque le build est terminé, d'autres actions peuvent être enclenchées :

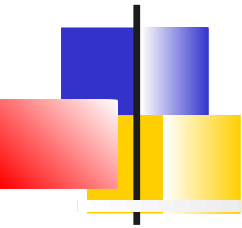
- Archiver les artefacts générés
- Créer des rapports sur l'exécution des tests
- Notifier l'équipe sur les résultats
- Démarrer un autre build
- Pousser une branche, tagger le SCM



Statuts d'un job

L'exécution d'un job peut avoir différents statuts :

- **SUCCESS** : Tout s'est bien passé. Actions du build et actions post-build
- **UNSTABLE** : Des tests, des seuils qualité ont échoués
- **FAILURE** : Des actions ou des post-actions ont échoués
- **NOT_BUILT** : Le job ne s'est pas exécuté
- **ABORTED** : Le job a été interrompu



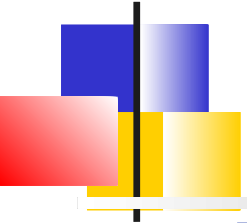
Pipelines

Approche et concepts

Syntaxe déclarative

Groovy et Syntaxe script

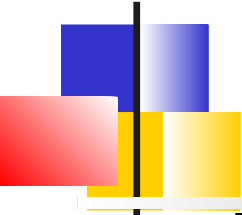
Librairies partagées



Introduction

Jenkins Pipeline est une suite de plugins qui permettent d'implémenter et d'intégrer des pipelines de CI/CD.

Grâce à Pipeline, les processus de build sont modélisés via du code et un langage spécifique (DSL)



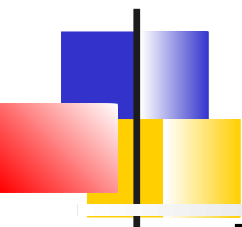
Définir une pipeline

Une Pipeline peut être créée :

- En saisissant un script directement dans l'interface utilisateur de Jenkins.
- En créant un fichier **Jenkinsfile** qui peut alors être committé dans le SCM.
Approche recommandée

Quelque soit l'approche, 2 syntaxes sont possibles :

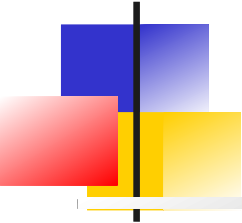
- Syntaxe déclarative
- Syntaxe script



Avantages de l'approche

Pipeline offre de nombreux avantages :

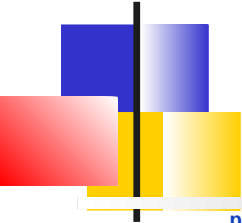
- Pipeline As Code : Les pipelines implémentées par du code sont gérées par le SCM
=> Historique des révisions, branches, ...
- Le DSL supporte des pattern de workflow complexes (fork/join, boucle, ...)
- Les Pipelines peuvent s'arrêter et attendre une approbation manuelle, elles survivent au redémarrage de Jenkins
- Il est possible de profiter de toute la puissance d'un langage de script comme Groovy en particulier de librairies partagées
- Les plugins Jenkins permettent d'étendre le DSL en proposant de nouvelles fonctions facilitant l'intégration



Termes du DSL

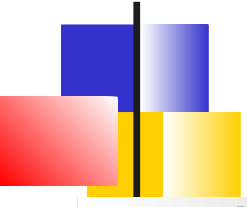
Le DSL introduit plusieurs termes et concepts :

- **node ou agent** : Les travaux d'une pipeline sont exécutés dans le contexte d'un nœud ou agent.
 - Plusieurs nœuds peuvent être déclarés dans une pipeline.
 - Chaque nœud a son propre espace de travail
- **stage (phase)** : Une phase définit un sous ensemble de la pipeline.
Par exemple : "Build", "Test", et "Deploy".
Les phases améliorent la compréhension et la visualisation de l'avancement de la pipeline
- **step (étape)** : Une tâche unitaire Jenkins.
Par exemple, exécuter un shell, publier les résultats de test



Exemple Jenkinsfile

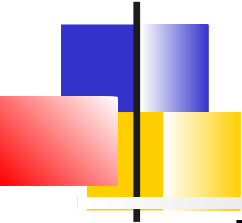
```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh './mvnw -Dmaven.test.failure.ignore=true clean test'
      }
      post {
        always { junit '**/target/surefire-reports/*.xml' }
      }
    }
    stage('Parallel Stage') {
      parallel {
        stage('Intégration test') {
          agent any
          steps {
            sh './mvnw clean integration-test'
          }
        }
        stage('Quality analysis') {
          agent any
          steps {
            sh './mvnw clean verify sonar:sonar'
          }
        }
      }
    }
  }
}
```



Jobs pipeline

Le plugin Pipeline ajoute de nouveaux types de Jobs :

- **Pipeline** : Définition d'une pipeline in-line ou dans un Jenkinsfile
- **Multi-branch pipeline** : On indique un dépôt et Jenkins scanne toutes les branches à la recherche de fichier Jenkinsfile. Un job est démarré pour chaque branche
- **Bitbucket/Team, Github** : On indique un compte et Jenkins scanne toutes les branches de tous les projet du serveur Bitbucket ou Github à la recherche de Jenkinsfile. Il démarre un job pour chaque Jenkinsfile trouvé

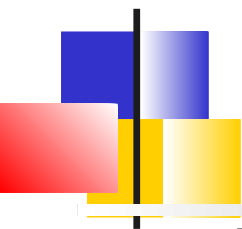


Variables d'environnement additionnelles

Les builds d'une pipeline multi-branches ont accès à des variables additionnelles :

- **BRANCH_NAME** : Nom de la branche pour laquelle la pipeline est exécutée
- **CHANGE_ID** : Identifiant permettant d'identifier le changement ayant provoqué le build

Les builds d'une pipeline multi-projet ont accès à des variables additionnelles identifiant le projet Github ou Bitbucket



Visualisation des pipelines

Le plugin ***Pipeline Graph View*** propose une visualisation dédiée aux pipelines :

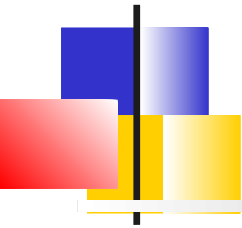
- Vue graphique des stages et de leur avancement
- Accès direct aux logs et au statut de chaque stage
- Intégration aux pipelines multi-branches

Il remplace Blue Ocean, déprécié depuis juillet 2026

Exemple : Détail d'un build

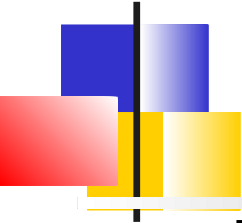
The image shows a screenshot of the Jenkins web interface. At the top, the Jenkins logo and navigation links (Pipelines, Administration, Déconnexion) are visible. Below this, the project name 'plbsi-angular' is displayed with a star and gear icon. The main section shows a successful build for 'plbsi-angular #59'. The build status is 'Succeeded' (indicated by a green checkmark). The build was triggered by 'Modifications par prosauvage' (Modifications by prosauvage). The build duration is '2m 43s' and it was last built '2 months ago'. The build is for the 'master' branch and commit '5c3e3ac'. Below the build information, a pipeline graph shows the build process: 'Init' (green checkmark) leads to 'Build' (green checkmark), which then branches into 'backend' (green checkmark) and 'frontend' (green checkmark), both of which lead to 'Integration' (green checkmark). At the bottom, the 'Étapes - Integration' (Steps - Integration) section lists the steps of the integration stage:

Step	Duration
Restore files previously stashed	<1s
Shell Script	27s
Shell Script	<1s
Shell Script	<1s
Shell Script	<1s
Shell Script	<1s
Shell Script	2s



Pipelines

Approche et concepts
Syntaxe déclarative
Groovy et Syntaxe script
Librairies partagées



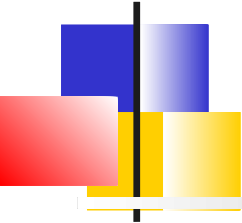
Documentation

La documentation est incluse dans Jenkins. Elle est accessible à :

`localhost:8080/pipeline-syntax/`

2 assistants permettent de générer des fragments de code

- ***Declarative Directive Generator*** n'est valable que pour la syntaxe déclarative
- ***Snippet Generator***, permettant de générer les steps, est valable pour les 2 syntaxes



Declarative Directive Generator

Sample Directive

agent: Agent

See [the online documentation](#) for more information on the agent directive.

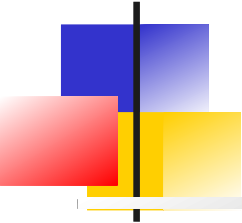
Agent

label: Run on an agent matching a label

Label

jdk8

Generate Declarative Directive



Snippet Generator

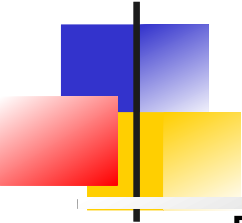
Sample Step

archiveArtifacts: Archive the artifacts

archiveArtifacts

Files to archive

Generate Pipeline Script



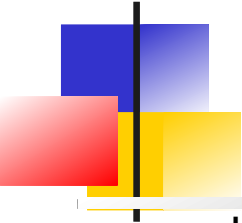
Références variables globales

En plus des générateurs, Jenkins fournit un lien vers le « Global Variable Reference » qui est également mis à jour en fonction des plugins installés.

Le lien documente les variables directement utilisables dans les pipelines

Par défaut, Pipeline fournit les variables suivantes :

- ***env*** : Variables d'environnement.
Par exemple : *env.PATH* ou *env.BUILD_ID*.
- ***params*** : Tous les paramètres de la pipeline dans une Map
Par exemple : *params.MY_PARAM_NAME*.
- ***currentBuild*** : Encapsule les données du build courant.
Par exemple : *currentBuild.result*, *currentBuild.displayName*



Généralités

La syntaxe déclarative est recommandée car plus simple d'accès et plus lisible.

Toutes les pipelines déclaratives sont dans un bloc **pipeline**.

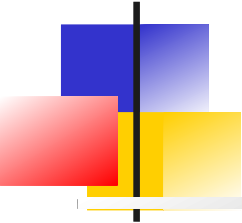
Elles contiennent toujours les sections suivantes :

- **stages** : Bloc contenant un ou plusieurs bloc stage
- **stage** : Bloc contenant un bloc steps et éventuellement un bloc post
- **steps** : Bloc contenant une succession de step unitaire

Des **directives**, par exemple agent, sont placées :

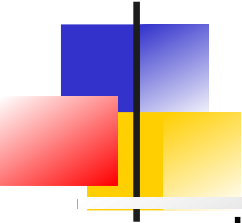
- soit au niveau pipeline
- soit au niveau d'un stage.

Éventuellement, des sections **post** spécifient les actions de post build



Structure

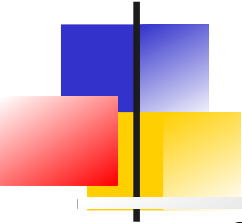
```
pipeline {  
  // Directives s'appliquant à toute la pipeline  
  agent any  
  stages {  
    stage('Compile et tests') {  
      // Directives ne s'appliquant qu'au stage  
      agent any  
      steps {  
        // Steps unitaires  
        echo 'Unit test et packaging'  
      } post { // Les actions de post-build  
        // Les différentes issues du build (stable, unstable, success, ...)  
      }  
    }  
    stage('Une autre phase') {  
      .....  
    }  
  }  
}
```



Section post

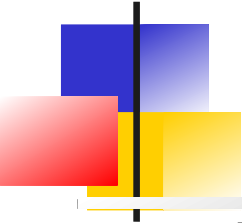
La section post définit une ou plusieurs steps qui sont exécutées en fonction du statut du build

- ***always*** : Étapes toujours exécutées
- ***changed*** : Seulement si le statut est différent du run précédent
- ***failure*** : Seulement si le statut est échoué
- ***success*** : Seulement si statut est succès
- ***unstable*** : Seulement si statut instable (Tests en échec, Violations qualité, porte perf., ..)
- ***aborted*** : Build avorté
- ...



Exemple

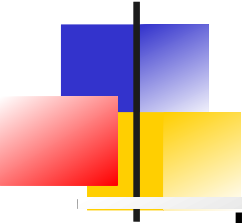
```
stage('Compile et tests') {  
    agent any  
    steps {  
        echo 'Unit test et packaging'  
        sh 'mvn -Dmaven.test.failure.ignore=true clean package'  
    }  
    post {  
        always {  
            junit '**/target/surefire-reports/*.xml'  
        }  
        success {  
            archiveArtifacts artifacts: 'application/target/*.jar', followSymlinks: false  
        }  
        failure {  
            mail bcc: '', body: 'http://localhost:8081/job/multi-branche/job/dev', cc: '',  
            from: '', replyTo: '', subject: 'Packaging failed', to: 'david.thibau@gmail.com'  
        }  
    }  
}
```



Directives

Les directives se placent soit :

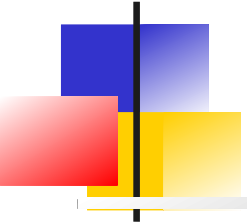
- Sous le bloc pipeline
=> Il s'applique à tous les stages de la pipeline
- Sous un bloc stage
=> Il ne s'applique qu'au stage concerné



Directive agent

La directive agent supporte les paramètres suivants :

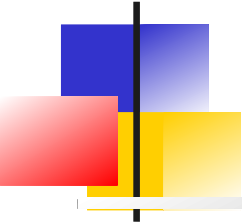
- **any** : N'importe quel agent.
- **label** : Agent ayant été labellisé par l'administrateur
- **none** : Permet de s'assurer qu'aucun agent ne sera alloué inutilement.
Placer au niveau global, force à définir un agent au niveau de stage
- **docker, dockerfile** : Si plugin docker présent
- **kubernetes** : Si plugin Kubernetes présent



Directive tools

tools permet d'indiquer les outils à installer sur l'exécuteur ou agent.

La section est ignorée si agent none Les outils supportés sont ceux installés par l'administrateur Jenkins



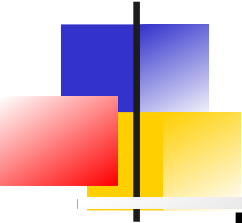
Directive tools

```
// agent avec le label jdk8
```

```
agent {  
  label 'jdk8'  
}
```

```
// Mise à disposition d'un outil sur l'agent
```

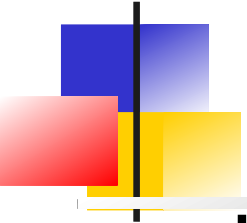
```
agent any  
tools {  
  maven 'Maven 3.5'  
}
```



Directive environment

La directive ***environment*** spécifie une séquence de paires clé-valeur qui seront définies comme variables d'environnement pour le stage.
La directive est souvent utilisée avec ***credentials()*** qui permet d'accéder aux crédits définis dans Jenkins.

```
pipeline {  
  agent any  
  environment {  
    NEXUS_CREDENTIALS = credentials('jenkins_nexus')  
    NEXUS_USER = "${env.NEXUS_CREDENTIALS_USR}"  
    NEXUS_PASS = "${env.NEXUS_CREDENTIALS_PSW}"  
  }  
}
```

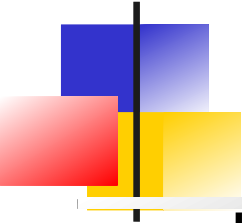


Directive options

La directive ***options*** permet de configurer des options globale à la pipeline.

Par exemple, timeout, retry, buildDiscarder, ..

```
pipeline {  
  agent any  
  options { timeout(time: 1, unit: 'HOURS') }  
  ...  
}
```

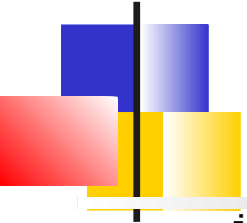


Directives input et parameters

La directive **input** permet de stopper l'exécution d'une pipeline et d'attendre une saisie manuelle d'un utilisateur

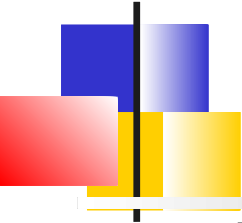
Via la directive **parameters**, elle peut définir une liste de paramètres à saisir. Chaque paramètre est défini par :

- Un type : String ou booléen, liste, ...
- Une valeur par défaut
- Un nom (Le nom de la variable disponible dans le script)
- Une description



Exemple directive input

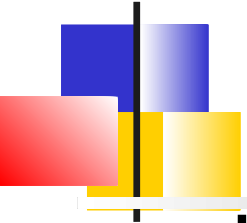
```
input {  
    message "Should we continue?"  
    ok "Yes, we should."  
    submitter "alice,bob"  
    parameters {  
        string(name: 'PERSON', defaultValue: 'Mr Jenkins', description: 'Who should  
        I say hello to?')  
    }  
}  
steps {  
    echo "Hello, ${PERSON}, nice to meet you."  
}
```



Directive triggers

La directive ***triggers*** définit les moyens automatique par lesquels la pipeline sera redéclenchée.

Les valeurs possibles sont *cron*, *pollSCM* et *upstream*

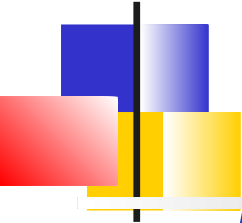


Directive when

La directive **when** permet à Pipeline de déterminer si le stage doit être exécutée

La directive doit contenir au moins une condition.

Si elle contient plusieurs conditions, toutes les conditions doivent être vraies. (Équivalent à une condition `allOf` imbriquée)



Conditions imbriquées disponibles

branch : Exécution si la branche correspond au pattern fourni

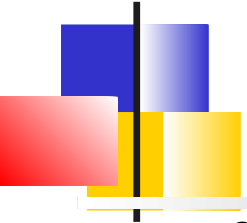
environnement : Si la variable d'environnement spécifié à la valeur voulue

expression : Si l'expression Groovy est vraie

not : Si l'expression est fausse

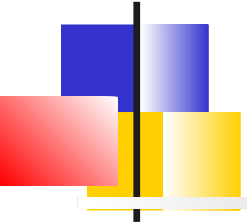
allOf : Toutes les conditions imbriquées sont vraies

anyOf : Si une des conditions imbriquées est vraie



Example

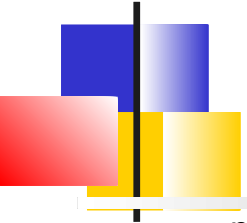
```
stage('Example Deploy') {  
  when {  
    allOf {  
      branch 'production'  
      environment name: 'DEPLOY_TO', value: 'production'  
    }  
  }  
  steps {  
    echo 'Deploying'  
  }  
}
```



Directive parallel

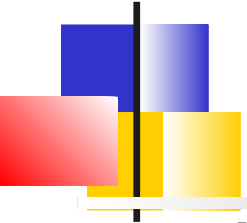
Avec la directive **parallel**, les stages peuvent déclarer des stages imbriqués qui seront alors exécutés en parallèle.

- Les stages imbriquées ne peuvent pas contenir à leur tour de stages imbriqués
- Le stage englobant ne peut pas définir d'agent ou de tools



Example

```
pipeline {  
  agent any  
  stages {  
    stage('Parallel Stage') {  
      when {branch 'master' }  
      parallel {  
        stage('Branch A') {  
          agent { label "for-branch-a" }  
          steps { echo "On Branch A" }  
        }  
        stage('Branch B') {  
          agent { label "for-branch-b"}  
          steps { echo "On Branch B" }  
        }  
      }  
    }  
  }  
}
```

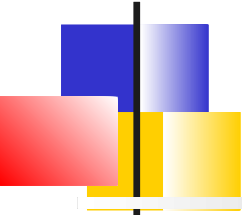


Steps disponibles

Les steps disponibles sont extensibles en fonction des plugins installés.

Voir la documentation de référence à :
<https://jenkins.io/doc/pipeline/steps/>

A noter que la version déclarative a une step ***script*** qui peut inclure un bloc dans la syntaxe script



Quelques steps

// Exécuter un script

sh, bat

// Archive the build output artifacts.

archiveArtifacts artifacts: 'output/*.txt', excludes:
 'output/*.md'

// Step basiques

stash, unstash : Mettre de côté puis reprendre
dir, deleteDir, pwd : Positionner le répertoire courant,
 supprimer un répertoire

fileExists, writeFile, readFile

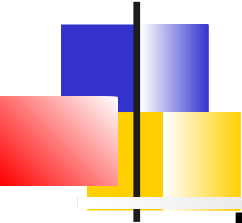
mail, git, build, error

sleep, timeout, waitUntil, retry

withEnv, credentials, tools

cleanWs() : Nettoyer le workspace

Voir : <https://jenkins.io/doc/pipeline/steps/>



Linter

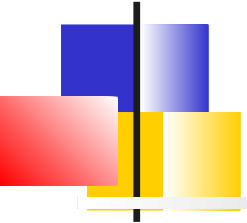
La syntaxe autorisée dans *Jenkinsfile* dépend des plugins installés sur le serveur.

Pour valider la syntaxe d'un *Jenkinsfile*, Jenkins propose un Linter accessible via Jenkins CLI ou l'API Rest

Exemple :

```
# ssh (Jenkins CLI)
# JENKINS_SSHD_PORT=[sshd port on controller]
# JENKINS_HOSTNAME=[Jenkins controller hostname]
ssh -p $JENKINS_SSHD_PORT $JENKINS_HOSTNAME declarative-linter < Jenkinsfile
```

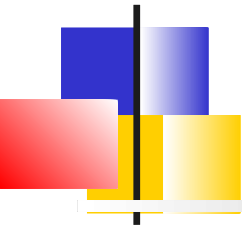
Des extensions d'IDE proposent une validation du *Jenkinsfile* via l'API Rest de Jenkins



Rejouer une pipeline

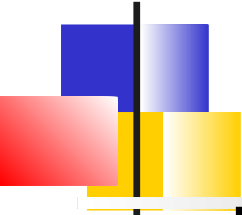
Sur un build exécuté, le lien "**Replay**" permet de le rejouer en apportant des modifications (sans changer la configuration de la pipeline et sans committer)

Après plusieurs essais, il est possible de récupérer les modifications pour les committer



Pipelines

Approche et concepts
Syntaxe déclarative
Groovy et Syntaxe script
Librairies partagées



Introduction

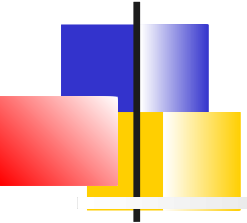
Une pipeline scriptée utilise directement la syntaxe Groovy.

- => La plupart des fonctionnalités du langage Groovy sont disponibles rendant l'outil très flexible et extensible

La syntaxe script n'est pas recommandée car moins lisible.

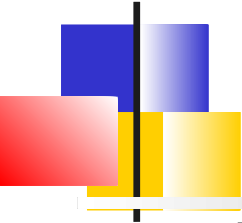
On l'utilise cependant assez souvent dans un bloc *script* à l'intérieur d'une pipeline déclarative

Une pipeline déclarative est en fait un script Groovy !



Exemple bloc script

```
pipeline {
  agent any
  stages {
    stage('Example') {
      steps {
        echo 'Hello World'
        script {
          def browsers = ['chrome', 'firefox']
          for (int i = 0; i < browsers.size(); ++i) {
            echo "Testing the ${browsers[i]} browser"
          }
        }
      }
    }
  }
}
```



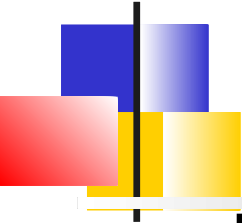
Déclaration de variables

En Groovy, il n'est pas nécessaire de préciser le type d'une variable lors de sa déclaration.

Il suffit d'utiliser le mot-clé ***def***

```
def x = 1
```

```
def s = 'Hello'
```



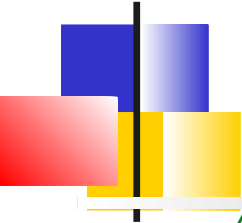
Script Groovy

Les scripts Groovy contiennent des instructions qui ne sont pas encapsulées par une déclaration de classe.

Ils peuvent également contenir des définitions de méthodes et des déclarations de variables

Un script est parsé intégralement avant son exécution

Le fichier Jenkinsfile est un script Groovy



Déclaration méthode et variables dans JenkinsFile

```
// Déclaration de variables globales
```

```
def integrationUrl
```

```
def dataCenters
```

```
pipeline {
```

```
    agent none
```

```
    .....
```

```
    stage {
```

```
        steps {
```

```
            script {
```

```
                checkSonarQualityGate()
```

```
            }
```

```
    }
```

```
// Définition d'une méthode
```

```
def checkSonarQualityGate() {
```

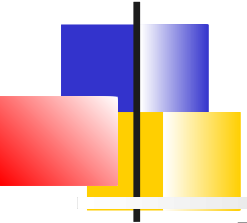
```
// Get properties from report file to call SonarQube
```

```
def sonarReportProps = readProperties file: 'target/sonar/report-task.txt'
```

```
def sonarServerUrl = sonarReportProps['serverUrl']
```

```
....
```

```
}
```

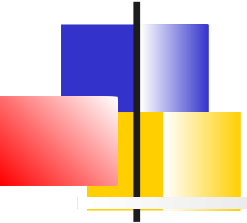


String

En Groovy, les littéraux String peuvent utiliser les simples ou double-quotes.

La version double-quotes permet l'utilisation d'expressions qui sont résolues à l'exécution.

```
def nick = 'ReGina'  
def book = 'Groovy in Action, 2nd ed.'  
assert "$nick is $book" == 'ReGina is Groovy in Action, 2nd ed.'
```

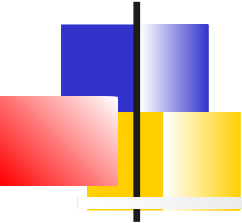


Collections

Groovy facilite la manipulation des collections en ajoutant des opérateurs, des instanciations via des littéraux

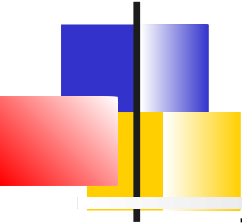
L'accès aux éléments est cohérent quelque soit le type de la collection (List, Map, ...)

Il introduit également un nouveau type : *Range* avec la notation ..



Exemples Collection List et Map

```
// roman est une Liste
def roman = ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII']
// On accède à un élément via son indice
assert roman[4] == 'IV'
// Redimensionnée automatiquement (pas d'ArrayIndexOutOfBoundsException)
roman[8] = 'VIII'
assert roman.size() == 9
// Les maps peuvent être facilement instanciées
def http = [
  100 : 'CONTINUE',
  200 : 'OK',
  400 : 'BAD REQUEST'
]
// Accès aux éléments via leur clé
assert http[200] == 'OK'
// Mise à jour d'une clé : Méthode put simplifiée
http[500] = 'INTERNAL SERVER ERROR'
assert http.size() == 4
```



Test et boucle

```
if (false) assert false // if sur une ligne
```

```
if (null) { // null est false  
    assert false  
}
```

```
// Boucle while classique
```

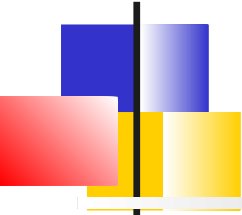
```
def i = 0  
while (i < 10) { i++ }  
assert i == 10
```

```
// for in sur un intervalle
```

```
def clicks = 0  
for (remainingGuests in 0..9) {  
    clicks += remainingGuests }  
assert clicks == (10*9)/2
```

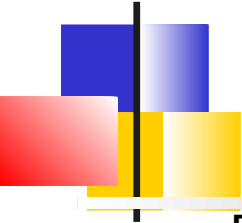
```
// for in sur une liste
```

```
def list = [0, 1, 2, 3]  
for (j in list) { assert j == list[j] }
```



Boucles sur les maps

```
def myMap = [a:1, b:2, c:3]
// Boucle sur les entries
def store = ''
myMap.each { entry ->
    store += entry.key
    store += entry.value
}
assert store == 'a1b2c3'
// Boucle sur les clés et valeurs
store = ''
myMap.each { key, value -> store += key ; store += value}
assert store == 'a1b2c3'
// Boucle sur les clés
store = ''
for (key in myMap.keySet()) {store += key}
assert store == 'abc'
// Boucle sur les valeurs
store = ''
for (value in myMap.values()) {store += value}
assert store == '123'
```



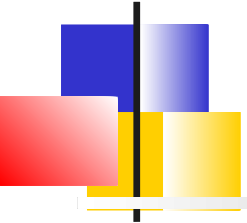
Définition de méthodes

En Java, une définition de méthode respecte la syntaxe suivante :

```
type nomMethode(type param1, type param2, ...) {  
    ...  
    return type  
}
```

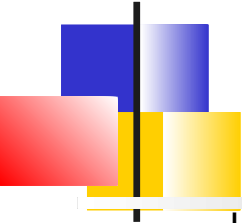
Groovy autorise quelques libertés :

- Déclarer un type de retour est optionnel (utilisation de def)
- L'instruction return est optionnelle. La dernière instruction est retournée.
- La déclaration explicite des types des paramètres est optionnelle.
- Les paramètres peuvent avoir une valeur par défaut
- Les appels de méthodes peuvent respecter l'ordre de déclaration des paramètres ou utiliser des paramètres nommés avec une Map



Map et paramètres nommés

```
class Summer {  
  
  def sumNamed(Map args){  
    ['a', 'b', 'c'].each{ println args.get(it,0)}  
    return args.a + args.b + args.c  
  }  
}  
  
def summer = new Summer()  
assert 2 == summer.sumNamed(a:1, b:1)
```



Parenthèses

Lors d'un appel de paramètres les parenthèses peuvent être omises :

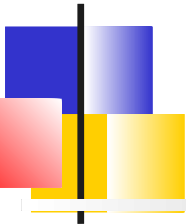
```
println "Hello"  
method a, b  
<=>  
println("Hello")  
method(a, b)
```

Dans le cas d'une closure :

```
list.each { println it }  
dependencies { }
```

Attention les parenthèses sont obligatoires lors d'appels imbriqués :

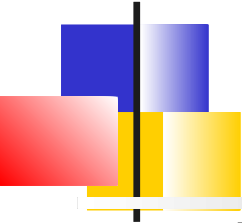
```
def foo(n) { n }  
def bar() { 1 }  
println foo 1 // Ne marche pas  
def m = bar // Ne marche pas
```



Application aux steps Jenkins

```
publishHTML([allowMissing: false, alwaysLinkToLastBuild: false,  
  icon: '', keepAll: false, reportDir: 'application/target',  
  reportFiles: 'dependency-check-report.html',  
  reportName: 'Analyse de dépendances OWASP',  
  reportTitles: '', useWrapperFileDirectly: true])
```

```
mail bcc: '', body: 'Pipeline en erreur', cc: '',  
  from: 'jenkins@plbformation.com',  
  replyTo: '', subject: 'Error !', to: 'david.thibau@gmail.com'
```

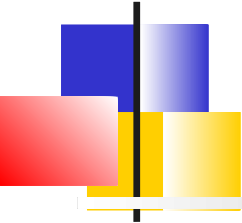


Closures

Les closures Groovy permettent la programmation fonctionnelle.

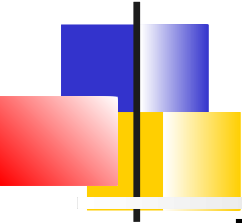
- Un bloc d'instructions peut être passé en paramètre à une méthode.

Un objet de type **Closure** travaille en sous-main



Exemple Jenkinsfile

```
steps {  
    echo 'Unit test et packaging'  
    sh 'mvn -Dmaven.test.failure.ignore=true clean package'  
}  
  
—  
  
withCredentials([string(credentialsId: 'NVD_API_KEY',  
    variable: 'NVD_API_KEY')]) {  
    sh 'mvn verify -Dnvd.api.key=$NVD_API_KEY -DskipTests'  
}
```



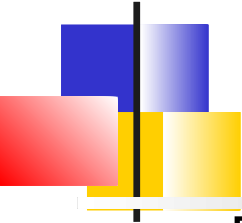
Objet délégué

La particularité des Closure est qu'il est possible de définir son objet délégué.
(résolution de this dans le bloc de la Closure)

Cette technique peut être utilisée par les DSL
comme ceux de Jenkins

```
// La méthode prend en argument une closure, (bloc d'instructions)
def call(body) {

    def config = [:]
    body.resolveStrategy = Closure.DELEGATE_FIRST
    body.delegate = config
    // les instructions sont exécutées mais l'objet courant (this) est la map config
    body()
}
```

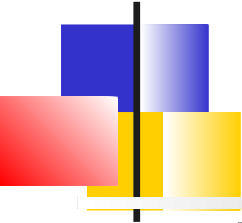


Full script : exécuteur

En mode full script, la fonction ***node()*** permet de provisionner un agent

```
// Un agent labellisé 'Windows'
node('Windows') {
  stage('Example') {
    if (env.BRANCH_NAME == 'master') {
      echo 'I only execute on the master branch'
    } else {
      echo 'I execute elsewhere'
    }
  }
}
```

A la différence de la directive agent, *node* ne provoque pas le clone du dépôt. Il faut utiliser un step ***checkout*** ou ***git***



Valeur de retour de sh

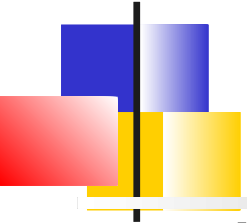
La step sh permet de récupérer la sortie standard ou l'exit status dans une variable :

// Positionner la clé de hash dans une variable

```
def gitCommit = sh(returnStdout: true,  
    script:'git rev-parse HEAD').trim()
```

// Statut du sh

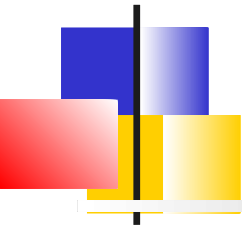
```
def result = sh (returnStatus: true,  
    script: "git log -1 | grep 'maven-release-plugin'" )
```



Plugin Utility Steps

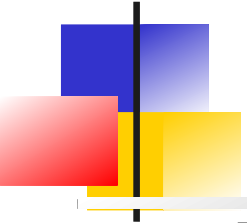
Le plugin ***Pipeline Utility Steps*** ajoute plein d'étapes utilitaires comme :

- Trouver un fichier dans le workspace
- Créer ou vérifier un SHA-1
- Créer des tar gz, des zip
- Lire des fichiers CSV, properties, YAML, JSON
- ...



Pipelines

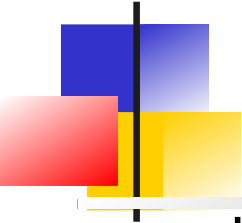
Approche et concepts
Syntaxe déclarative
Groovy et Syntaxe script
Librairies partagées



Introduction

Pipeline permet la création de librairies partagées pouvant être définies dans des dépôts de sources externes et chargées lors de l'exécution d'une Pipeline.

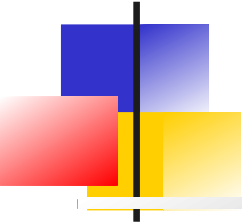
Une librairie est constituée de fichiers Groovy



Étapes de mise en place

La mise en place consiste en :

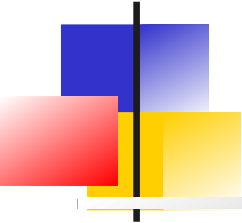
- 1) Créer les scripts groovy en respectant une arborescence projet.
Committer dans un dépôt
- 2) Spécifier la librairie dans Jenkins
Manage Jenkins → System → Global Pipeline Libraries :
 - Un nom
 - Une méthode de récupération (scm)
 - Une version par défaut
- 3) L'importer dans un projet en utilisant l'annotation
@Library



Code groovy

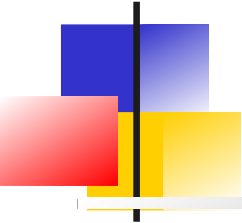
Différents types de codes peuvent être développés dans une librairie :

- Classes groovy classique, définissant des structures de données, des méthodes.
Pour les utiliser, il faudra les instancier ;
Pour interagir avec les variables de la pipeline, il faudra les passer en paramètre.
Utilisable dans pipeline script
- Définir des variables globales. Jenkins les instancie automatiquement comme singleton et elles apparaissent dans l'aide.
Utilisable dans pipeline script
- Définir des nouvelles steps. Idem variable globale + mise à disposition de la méthode call()
Dans ce cas, utilisable en script et déclaratif



Structure projet

```
(root)
+- src                                # Classes Groovy classiques
|   +- org
|       +- foo
|           +- Bar.groovy           # Classe org.foo.Bar
+- vars
|       +- foo.groovy               # Variable globale 'foo'
|       +- foo.txt                  # Aide pour la variable 'foo'
+- resources                          # Fichiers ressources
|   +- org
|       +- foo
|           +- bar.json             # Données pour org.foo.Bar
```



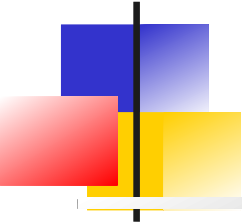
Exemple Code classique

Fichier *src/org/foo/Zot.groovy*

```
package org.foo
def checkOutFrom(repo) {
    git url: "git@github.com:jenkinsci/${repo}"
}
```

Utilisation dans une pipeline

```
script {
    def z = new org.foo.Zot()
    z.checkOutFrom('myRepo')
}
```



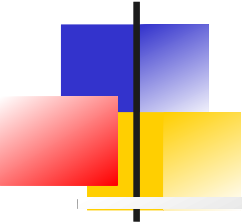
Variable globale

Fichier ***vars/log.groovy***. Le nom du fichier doit être en camelCase.

```
def info(message) {  
    echo "INFO: ${message}"  
}  
def warning(message) {  
    echo "WARNING: ${message}"  
}
```

Utilisation dans une pipeline

```
pipeline {  
    ...  
    script {  
        log.info 'Starting'  
        log.warning 'Nothing to do!'  
    } } }
```



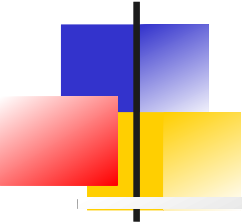
Nouvelle step

Fichier **vars/sayHello.groovy**.

```
def call(String name = 'human') {  
    echo "Hello, ${name}."  
}
```

Utilisation dans une pipeline

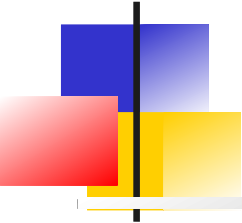
```
pipeline {  
    ...  
    steps {  
        sayHello 'Joe'  
    } } }
```



Définition de librairies

Les librairies une fois développées peuvent être installées de différentes façons :

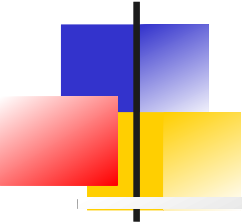
- **Global** Pipeline Libraries :
Manage Jenkins → System → Global Pipeline Libraries
- **Folder** : Une librairie peut être définie au niveau d'un dossier
- **Certains plugins** ajoutent des façons de définir des librairies.
Ex : *github-branch-source*



Chargement des librairies

Les librairies doivent être chargées pour être utilisées.
Plusieurs techniques possibles :

- L'administrateur a marqué la librairie comme « ***Load Implicitly*** »
- Le Jenkinsfile charge explicitement la librairie en utilisant
 - l'annotation ***@Library*** en début de fichier
 - La step ***library*** qui charge dynamiquement la librairie.



Option Load Implicitly

Global Pipeline Libraries

Sharable libraries available to any Pipeline jobs running on this system. These libraries will be trusted, meaning they run without "sandbox" restrictions and may use @Grab.

Library

Name

my-shared-library

?

Default version

master

?

Load implicitly

☐

?

Allow default version to be overridden

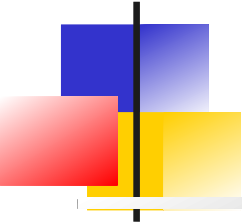
☒

?

Retrieval method

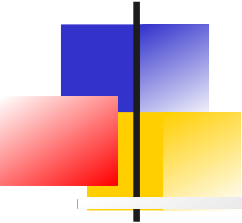
☒ Modern SCM

?



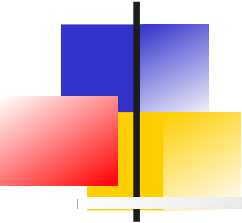
Annotations

```
@Library('my-shared-library') _  
/* Avec une version, branch, tag, ou autre */  
@Library('my-shared-library@1.0') _  
/* Plusieurs librairies */  
@Library(['my-shared-library', 'otherlib@abc1234'])  
/* Typiquement devant la classe importée */  
@Library('somelib') _  
import com.mycorp.pipeline.somelib.UsefulClass
```



Step library

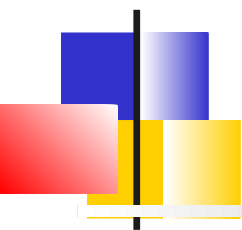
```
library identifier: 'custom-lib@master',  
retriever: modernSCM( [$class: 'GitSCMSource',  
remote: 'git@git.mycorp.com:my-jenkins-  
utils.git', credentialsId: 'my-private-key'] )
```



Librairies de tiers

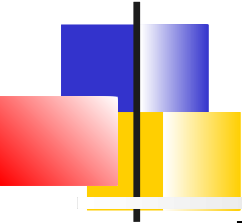
Il est possible également de charger une librairie à partir du dépôt Maven Central en utilisant l'annotation **@Grab**

```
@Grab('org.apache.commons:commons-math3:3.4.1')
import org.apache.commons.math3.primes.Primes
void parallelize(int count) {
    if (!Primes.isPrime(count)) {
        error "${count} was not prime"
    }
    // ...
}
```



Jenkins et les containers

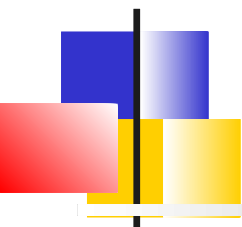
Docker
Kubernetes



Jenkins et Docker

Plusieurs cas d'usage de Docker dans un contexte Jenkins :

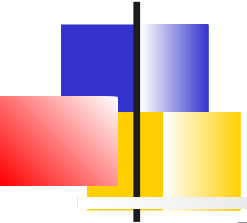
- Utiliser des images pour exécuter les builds
- Construire et pousser des images pendant l'exécution d'une pipeline
- Utiliser des images pour exécuter des services nécessaires à une étape de build (Démarrer un serveur lors de test d'intégration/fonctionnel)
- Dockeriser des configurations Jenkins



Docker pipeline plugin

Docker pipeline permet d'utiliser des containers pour exécuter les steps de la pipeline via la **directive agent**

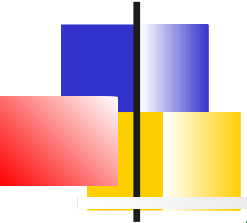
Il met également à disposition une variable ***docker*** permettant de construire et publier des images



Agent Docker

Pipeline permet une utilisation facile des images Docker comme environnement d'exécution pour un Stage ou pour toute la Pipeline.

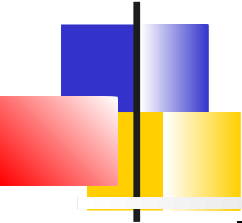
Il suffit de préciser l'attribut image à la directive agent



Exemple

```
// Declarative //
pipeline {
  agent {
    docker { image 'node:22-alpine' }
  }
  stages {
    stage('Test') {
      steps { sh 'node --version' }
    }
  }
}

// Script //
node {
  docker.image('node:22-alpine').inside {
    stage('Test') { sh 'node --version' }
  }
}
```

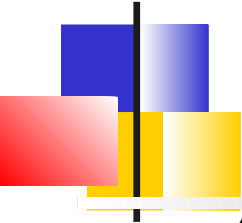


Gestion de cache

Les outils build téléchargent généralement les dépendances externes et les stocke localement pour les réutiliser.

- Pour réutiliser les téléchargements entre 2 builds, il faut monter des volumes persistants sur les nœuds exécutant les builds

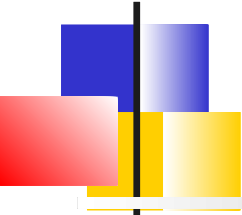
Le bloc *docker{}* permet de passer des arguments à la commande de démarrage du container



Exemple

```
// Declarative //
pipeline {
  agent {
    docker {
      image 'maven:3.9-eclipse-temurin-21'
      args '-v $HOME/.m2:/root/.m2'
    }
  }
  stages {
    stage('Build') {
      steps { sh 'mvn -B clean' }
    }
  }
}

// Script //
node {
  docker.image('maven:3.9-eclipse-temurin-21').inside('-v $HOME/.m2:/root/.m2') {
    stage('Build') { sh 'mvn -B' }
  }
}
```



Dockerfile

Pipeline permet également de construire des images à partir d'un Dockerfile du dépôt de source.

Il faut utiliser la syntaxe déclarative :

```
// Declarative //
pipeline {
  agent { dockerfile true }
  stages {
    stage('Test') { steps {sh 'node --version'} }
  }
}
```

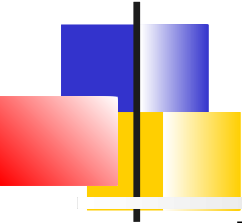
Docker Label

Par défaut, Jenkins suppose que tous les agents sont capables d'exécuter une pipeline Docker, ce qui peut poser problème si certains agents ne peuvent pas exécuter le daemon Docker.

Le plugin *Docker Pipeline* fournit une option globale permettant de spécifier un label pour les agents acceptant Docker

Pipeline Model Definition

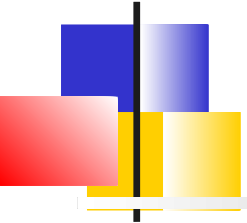
Docker Label	
Docker registry URL	
Registry credentials	- none - Add



Construction d'image

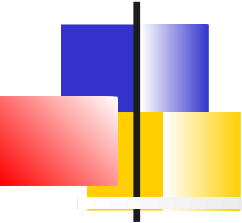
Le plugin Docker Pipeline met à disposition la variable ***docker*** qui permet entre autres de :

- Déclarer un registre
- Construire ou récupérer une image
- Tagger, Pousser, Tirer des images
- Découvrir le mapping du port d'un conteneur en exécution
- ...



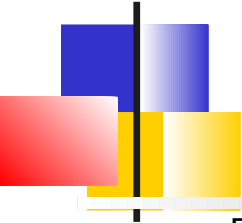
Exemple

```
script {  
  def dockerImage = docker.build('dthibau/multi-module', '.')  
  docker.withRegistry('https://registry.hub.docker.com',  
    'dthibau_docker') {  
    dockerImage.push "${BRANCH_NAME}"  
  }  
}
```



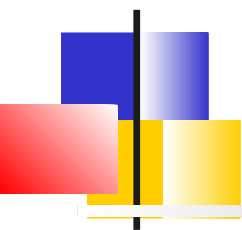
Exemple avancé – side car pattern

```
node {
  checkout scm
  docker.image('mysql:8').withRun('-e "MYSQL_ROOT_PASSWORD=my-secret-pw"') {
    c ->
    docker.image('mysql:8').inside("--link ${c.id}:db") {
      /* Wait until mysql service is up */
      sh 'while ! mysqladmin ping -hdb --silent; do sleep 1; done'
    }
    docker.image('debian:12').inside("--link ${c.id}:db") {
      /*
       * Execution de tests nécessitant MySQL,
       * Disponible sur le host `db`
       */
      sh 'make check'
    }
  }
}
```



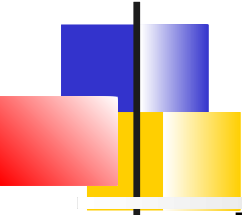
Exemple customisation

```
FROM jenkins/jenkins:lts-jdk21
USER root
RUN apt-get update && apt-get install -y ca-certificates curl gnupg
RUN install -m 0755 -d /etc/apt/keyrings && \
curl -fsSL https://download.docker.com/linux/debian/gpg \
-o /etc/apt/keyrings/docker.asc
RUN echo "deb [arch=amd64 \
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/debian bookworm stable" \
> /etc/apt/sources.list.d/docker.list
RUN apt-get update && apt-get install -y docker-ce-cli
USER jenkins
RUN jenkins-plugin-cli --plugins "pipeline-graph-view docker-workflow"
```



Jenkins et les containers

Docker
Kubernetes



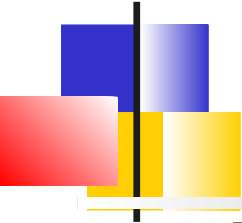
Kubernetes plugin

Le plugin Kubernetes fournit des agents **éphémères** :
un pod est créé pour chaque build, puis supprimé

Ce que cela change :

- Aucun agent permanent à installer ni à maintenir
- Chaque build part d'un environnement propre : plus d'effet de bord entre builds
- La capacité suit la charge : zéro agent au repos, N pods en pic
- Les outils sont versionnés dans des images, pas installés sur des machines

Contrepartie : tout ce qui n'est ni dans une image ni dans un volume disparaît avec le pod

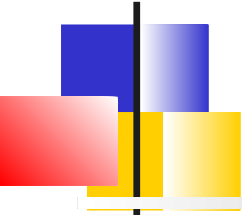


Anatomie du pod d'agent

Pod créé pour un build

- └─ conteneur jnlp *toujours présent*
 - | jenkins/inbound-agent
 - | se connecte au contrôleur et pilote les steps
- └─ conteneur maven *vos outils*
- └─ conteneur node
- └─ volume workspace (emptyDir)
 - | monté dans tous les conteneurs

Le workspace est partagé : un fichier produit dans maven est visible dans node



Pod Template

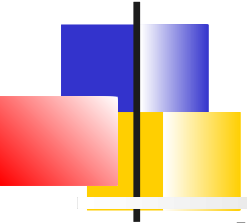
Un **podTemplate** décrit le pod qui exécutera le build

Trois façons de le fournir :

- Défini par l'administrateur dans le cloud, repris par ***inheritFrom***
- Écrit dans le Jenkinsfile : ***yaml*** en ligne ou ***yamlFile***
- Hérité puis surchargé : inheritFrom + ***yamlMergeStrategy*** merge() ou override()

Options utiles de la directive kubernetes :

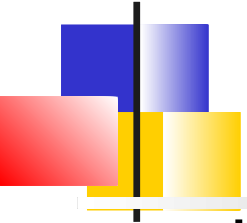
- ***defaultContainer*** : conteneur par défaut des steps, évite de répéter container()
- ***idleMinutes*** : garde le pod quelques minutes pour le build suivant



Exemple : podTemplate nommé

Utilisation podTemplate nommé default :

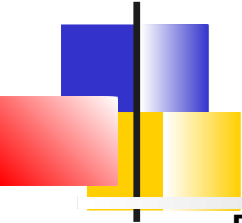
```
agent {  
  kubernetes {  
    inheritFrom 'default'  
  }  
}
```



Exemple : plusieurs conteneurs

Utilisation d'un podTemplate contenant plusieurs containers dont jdk21

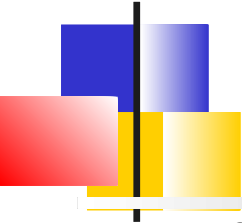
```
agent {  
  kubernetes {  
    inheritFrom 'jdk21-agent'  
  }  
}  
steps {  
  container(name: 'jdk21') {  
    sh 'javac -version'  
  }  
}
```



Exemple : redéfinition complète

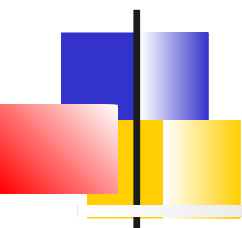
Redéfinition du podTemplate à utiliser

```
pipeline {
  agent {
    kubernetes {
      yamlFile 'KubernetesPod.yaml'
    }
  }
  stages {
    stage('Compile') {
      steps {
        container(name: 'jdk21') {
          sh 'javac -version'
        }
      }
    }
    ...
  }
}
```



Exemple yaml

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    some-label: some-label-value
spec:
  containers:
    - name: jdk21
      image: eclipse-temurin:21-jdk
      command:
        - cat
      tty: true
    - name: busybox
      image: busybox
      command:
        - cat
      tty: true
```



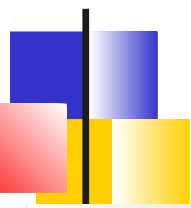
Garder le conteneur vivant

command: ["cat"] et **tty: true** dans chaque conteneur outil :

- Un conteneur s'arrête dès que son processus principal se termine
- cat, maintenu par un pseudo-terminal, garde le conteneur en vie
- Jenkins y exécute ensuite les steps via l'API exec de Kubernetes
- Inutile pour une image dont l'entrypoint tourne déjà en continu

resources : à renseigner systématiquement

- requests : ce que le scheduler réserve — sans requests, le pod atterrit n'importe où
- limits : plafond — un dépassement mémoire fait tuer le conteneur (OOMKilled)
- Pour la JVM : -XX:MaxRAMPercentage plutôt qu'un -Xmx fixe



Volumes, workspace et cache

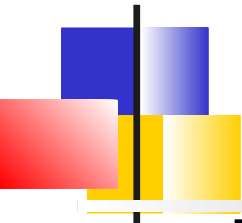
Le workspace est un emptyDir : il naît et meurt avec le pod

Conséquence directe : le cache de dépendances est vide à chaque build

Réponses possibles :

- ***persistentVolumeClaim*** monté sur ~/.m2 ou ~/.npm — attention à l'accès concurrent
- ***Dépôt proxy*** (Nexus, Artifactory) dans le cluster — pas d'état partagé, et utile aussi aux développeurs
- ***Image d'agent pré-peuplée*** pour les dépendances stables

Autres volumes : secretVolume, configMapVolume, hostPathVolume (à éviter)



Construire une image dans un pod

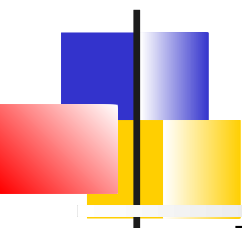
Dans un pod il n'y a pas de démon Docker : **`docker.build()`** du Docker Pipeline plugin ne fonctionne pas

Monter `/var/run/docker.sock` du nœud revient à donner le root du nœud au build — à proscrire

Options :

- **BuildKit** rootless (moby/buildkit) — le plus activement développé
- **Buildah** en mode rootless
- **Kaniko** — dépôt Google archivé en juin 2025, repris par Chainguard ; les images `gcr.io/kaniko-project` ne sont plus mises à jour

Dans tous les cas : un conteneur dédié dans le podTemplate, et les credentials du registre montés en secretVolume



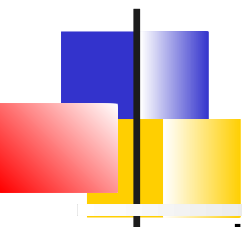
Cycle de vie et placement

Durée de vie du pod

- ***idleMinutes*** : garde le pod N minutes pour le build suivant — gros gain sur des builds qui s'enchaînent
- ***podRetention*** : never() / onFailure() / always() — onFailure garde le pod pour l'autopsie
- ***activeDeadlineSeconds*** : arrête un build qui traîne

Placement

- ***nodeSelector*** et ***tolerations*** : cantonner les builds sur un pool de nœuds dédié
- Avec un autoscaler sur ce pool, les nœuds suivent la charge des pods



Diagnostic

Le build reste en attente d'un exécuteur

- `kubectl get pods -n <namespace>` puis `kubectl describe pod`
- **Pending** : requests trop élevées ou nœuds saturés
- **ImagePullBackOff** : image introuvable ou credentials de registre manquants

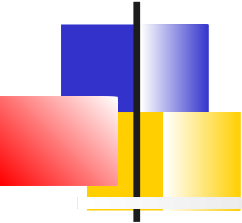
L'agent ne se connecte jamais

- Vérifier Jenkins URL, Jenkins tunnel, et que le port 50000 est joignable depuis le pod

Le conteneur s'arrête immédiatement

- `command` et `tty` manquants dans le `podTemplate`

Logs de l'agent : `kubectl logs <pod> -c jnlp`



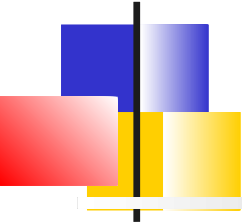
Références

Jenkins User Handbook –
www.jenkins.io/doc/book

Référence des steps –
www.jenkins.io/doc/pipeline/steps

Index des plugins – plugins.jenkins.io

Forum communautaire –
community.jenkins.io



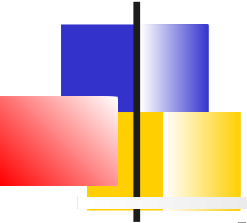
Annexes

Tests et analyse statique

Infrastructure as Code

Le plugin JTE

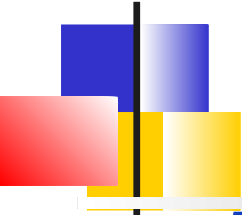
Syntaxe script



Plugin junit

Le plugin *junit* permet de publier les résultats des tests.

- S'applique à tous les outils fournissant des résultats au format XML de junit
- Fournit une visualisation graphique des résultats de test, des historiques ainsi qu'une interface utilisateur Web pour afficher les rapports de test, suivre les échecs, etc.



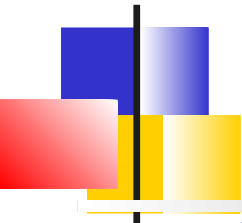
Paramètres de JUnit

Test report XMLs : Plusieurs chemins vers les fichiers XML de résultat séparés par des espaces. Syntaxe Ant

Retain long standard output/error : Si coché, cela permet de conserver les sorties standard et d'erreur complètes des tests.

Health report amplification factor : Comment les résultats des tests affectent le statut du build.
Le facteur par défaut est 1,0. Un facteur de 0,0 désactivera la contribution des résultats de test à l'établissement du score de santé et, à titre d'exemple, un facteur de 0,5 signifie que 10 % des tests échoués obtiendront 95 % de santé.

Allow empty results : Si cochée, des résultats vides ne font pas échouer le build.



Paramètres de JUnit (2)

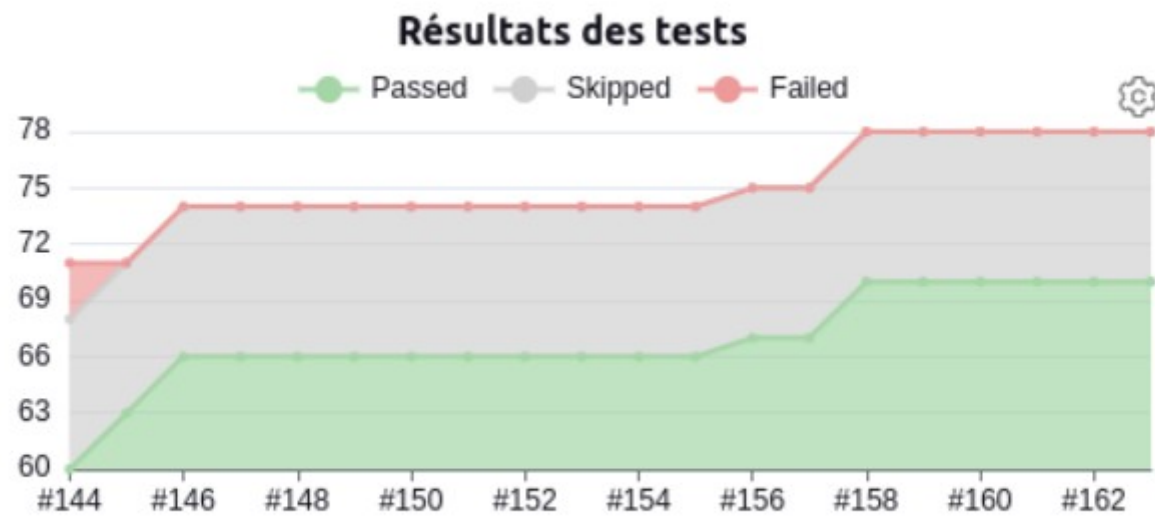
Skip publishing checks for failed builds: Si cochée, les rapports ne sont pas publiés si le build échoue.

Check name: Le nom utilisé lors de la publication des résultats . Le nom peut également être surchargé avec *withChecks*

```
stage('Ignored') {  
    withChecks('Integration Tests') {  
        junit 'yet-more-test-results.xml'  
    }  
}
```

Skip marking build unstable: Si pas coché le build devient instable si au moins un échec. Si coché, le build est en succès même si il y a des erreurs

Graphe de tendance



Résultats des tests

1 échecs (±0) , 8 non passés (±0)

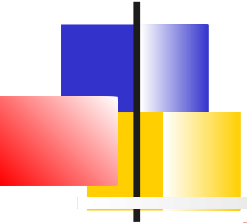
92 tests (±0)
A pris 5 mn 58 s.
Ajouter une description

Tous les tests qui ont échoué

Nom du test	Durée	Age
com.plb.plbsiapi.offre.gescof.GescofSynchroServiceTest.testCheckFamilleProduit Error Details 403 Forbidden: [{"code":403,"message":"Vous n'avez pas les droits suffisants pour effectuer cette action."}] + Stack Trace + Standard Output	0.66 s	8

Tous les tests

Package	Durée	Échec	(diff)	Sauté	(diff)	Pass	(diff)	Total	(diff)
com.plb.plbsiapi	8 ms	0		0		1		1	
com.plb.plbsiapi.cms	0 ms	0		0		1		1	
com.plb.plbsiapi.core.service	1.3 s	0		0		2		2	
com.plb.plbsiapi.elk.offre.service	1.8 s	0		0		2		2	



Plugin Performance

Performance Plugin permet :

- d'exécuter des tests de performances lors d'une étape de construction avec l'outil Taurus
- de créer des rapports à partir de fichiers de résultats de tests préexistants.
Supporte les formats de Taurus, JMeter, HPE LoadRunner, wrk

[Back to Dashboard](#)

[Status](#)

[Changes](#)

[Workspace](#)

[Build Now](#)

[Delete Project](#)

[Configure](#)

[Performance Trend](#)

Build History [\(trend\)](#)

- [#10 Mar 22, 2010 10:36:48 AM](#)
- [#9 Mar 22, 2010 9:59:28 AM](#)
- [#8 Mar 22, 2010 9:46:45 AM](#)
- [#7 Mar 22, 2010 9:38:15 AM](#)
- [#6 Mar 9, 2010 1:22:57 PM](#)
- [#5 Mar 9, 2010 12:09:36 PM](#)
- [#3 Mar 9, 2010 11:06:32 AM](#)
- [#2 Mar 9, 2010 10:47:59 AM](#)
- [#1 Mar 9, 2010 10:38:19 AM](#)

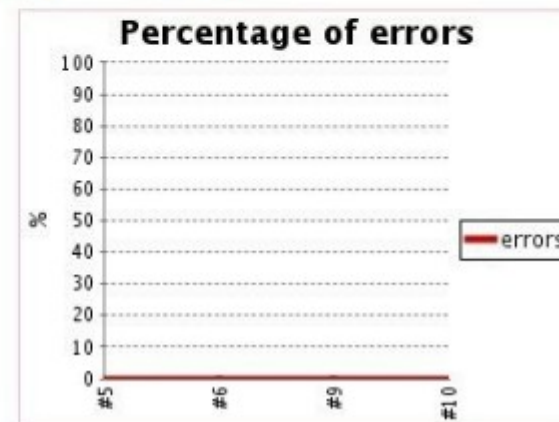
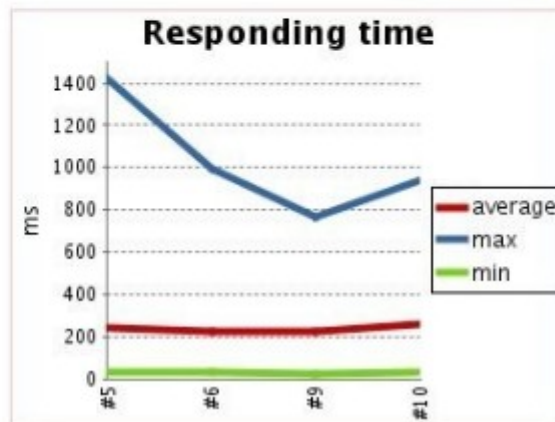
[for all](#) [for failures](#)

Performance Trend

[Last Report](#)

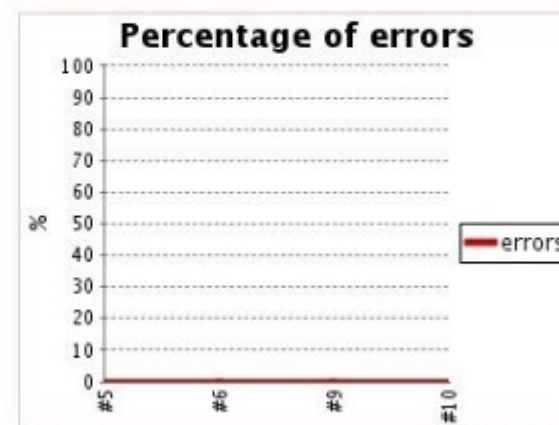
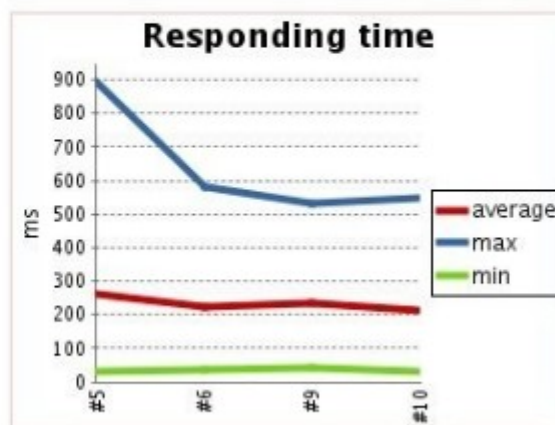
[Filter trend data](#)

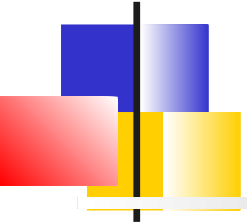
Test file: myTests1.jtl



[Trend report](#)

Test file: myTests2.jtl



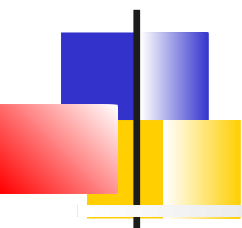


Autres plugins

Cucumber Report : Publication des rapports de tests Cucumber

Robot Framework Plugin : Publie les résultats de Robot

Selenium Plugin : Publie les résultats Selenium

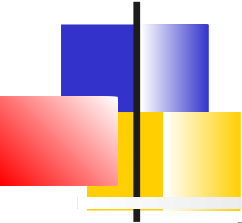


Plugin SonarQube Scanner

Le plugin ***SonarQube Scanner*** permet de centraliser la configuration de SonarQube dans Jenkins (Serveurs et Analyseurs)

L'analyse d'un projet peut alors être définie comme étape d'un build.

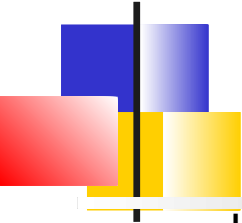
Une fois l'analyse terminée, un statut de qualité est remonté sur l'UI Jenkins et un lien permet d'accéder aux tableaux de bord SonarQube



Configuration

La mise en place consiste à :

- Définir l' ou les instances de SonarQube
Manage Jenkins → System → SonarQube servers → Add SonarQube
- Définir les scanners à utiliser
Manage Jenkins → Tools → SonarQube Scanner
- Configurer un projet pour utiliser le scanner dans une étape de build



Steps du plugin Sonar

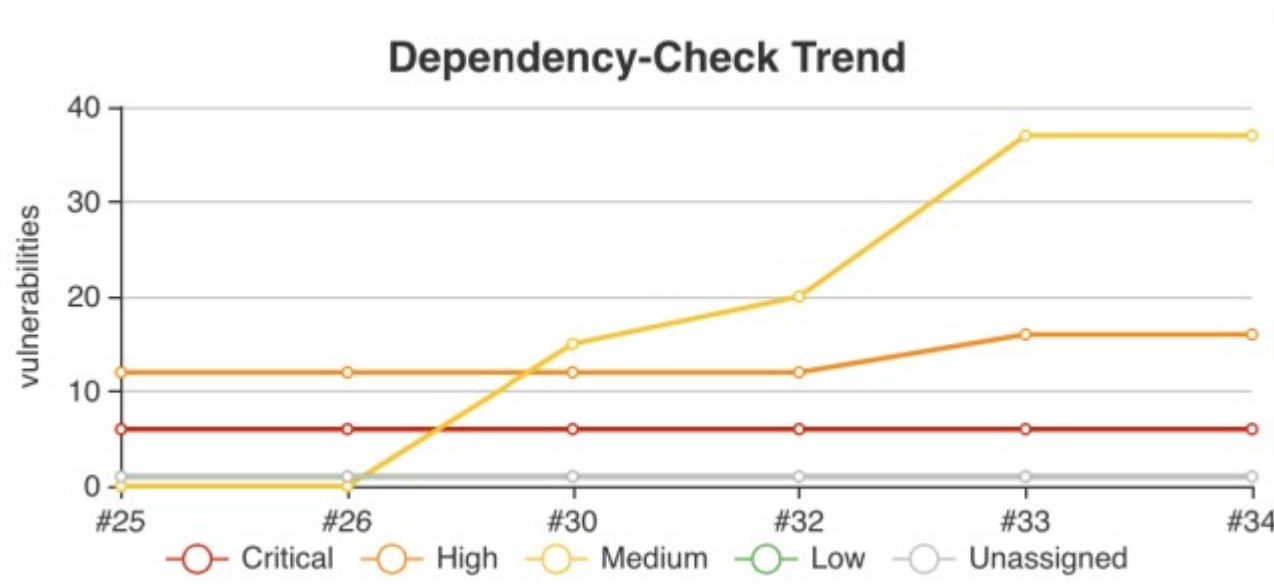
Le plugin ajoute des DSLs

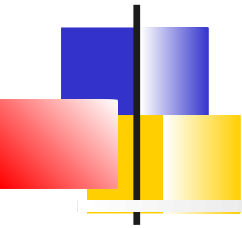
```
// Positionne les variables d'environnement
// SONAR_CONFIG_NAME, SONAR_HOST_URL, SONAR_AUTH_TOKEN .
withSonarQubeEnv('My SonarQube Server', envOnly: true) {
    println ${env.SONAR_HOST_URL}
}
// Pause pour récupérer le statut de la porte qualité
stage("Quality Gate") {
    steps {
        timeout(time: 1, unit: 'HOURS') {
            // true = pipeline est UNSTABLE si la porte qualité échoue
            waitForQualityGate abortPipeline: true
        }
    }
}
```

Vulnérabilités

Le plugin OWASP Dependency-Check permet :

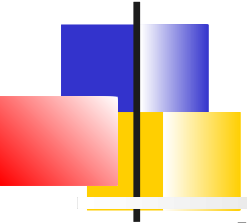
- D'exécuter des tests de vulnérabilités
- De publier des tests déjà présents





Annexes

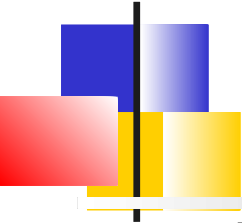
Tests et analyse statique
Infrastructure as Code
Le plugin JTE
Syntaxe script



Ansible

Le ***plugin Ansible*** permet d'exécuter des tâches Ansible en tant qu'étape du build.

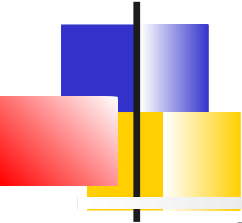
Ansible doit être dans le PATH
(déclaration dans les outils)



Commandes Ad Hoc

Les commandes ad hoc permettent d'effectuer des opérations simples sans écrire un playbook complet.

```
ansibleAdhoc(credentialsId: 'private_key',  
inventory: 'inventories/a/hosts', hosts:  
'hosts_pattern', moduleArguments:  
'module_arguments')
```



Playbook

Les playbook Ansible peuvent s'exécuter avec le plugin. Cela apporte :

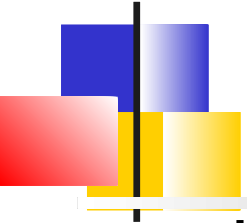
- Utilisation de crédentiel Jenkins
- une sortie couleur sans tampon dans le journal
- Utilisation des variables d'environnement Jenkins dans les playbooks

Exemple

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo "Compilation / tests"
      }
    }

    stage('Deploy with Ansible') {
      steps {
        ansiblePlaybook(
          playbook: 'deploy.yml',           // Playbook à exécuter
          inventory: 'inventories/prod/hosts', // Inventaire
          credentialsId: 'ssh-key-prod',     // Credentials Jenkins
          colorized: true,                  // Logs colorisés
          extraVars: [
            version: "${env.BUILD_NUMBER}", // Variables passées à Ansible
            env: "production"
          ]
        )
      }
    }
  }
}
```



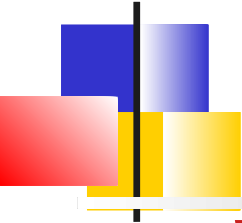
Vault

La plupart des opérations Ansible Vault peuvent être effectuées avec le plugin.

- Les opérations interactives telles que la création, la modification et l'affichage ne sont pas prises en charge via le plugin.

Permet aux développeurs de crypter des valeurs secrètes tout en gardant secret le mot de passe du vault.

```
ansibleVault(action: 'encrypt',  
  input:'vars/secrets.yml',  
  vaultCredentialsId:'ansible_vault_credentials')
```



Autres plugins

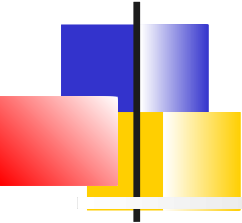
VMware vSphere Plugin: Création et gestion de machine virtuelle

Vagrant Plugin : Idem

Plugins fournisseur de cloud :

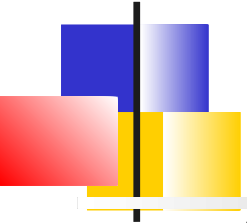
- ***AWS Step Plugin***
- ***Google Cloud Build Plugin***
- ***DigitalOcean Plugin***

Kubernetes CLI plugin permet de configurer kubectl



Annexes

Tests et analyse statique
Infrastructure as Code
Le plugin JTE
Syntaxe script



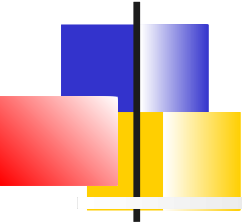
Principes

Au lieu que chaque projet d'une entreprise passe du temps à développer sa propre pipeline.

Une équipe transverse fournit :

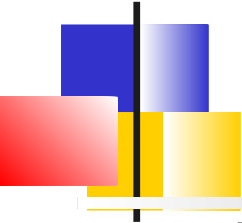
- Un gabarit de pipeline définissant les phases nécessaires à la construction d'un projet
- Des bibliothèques qui implémentent ces phases pour les différents outils de l'entreprise.

Les équipes projets n'ont plus qu'à fournir un fichier de configuration qui spécifie les bibliothèques à utiliser pour son projet.



Exemple Gabarit

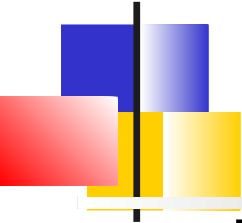
```
on_pull_request to: develop, {  
  continuous_integration()  
}  
on_merge to: develop, {  
  continuous_integration()  
  deploy_to dev  
  penetration_test()  
  integration_test()  
  performance_test()  
}  
on_merge to: main, {  
  deploy_to prod  
}
```



Librairies

Les librairies sont poussées dans un dépôt séparé respectant une structure de fichiers Groovy

```
exampleLibraryName #
├── steps #
│   ├── step1.groovy # Implémentation de la step1
│   └── step2.groovy
├── resources # Fichiers ressources accessibles par les steps
│   ├── someResource.txt #
│   └── nested
│       └── anotherResource.json #
├── src #
│   └── example
│       └── Utility.groovy # Classe utilitaire utilisable par les steps
└── library_config.groovy # Permet de valider l'utilisation de la lib
```



Step.groovy

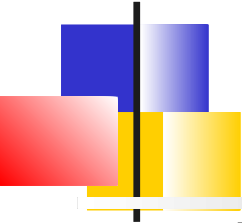
Par défaut, le nom de l'étape est basé sur le nom du fichier sans l'extension .groovy

Elle doit implémenter la méthode
void call() {}
qui peut prendre des paramètres¹

Des variables sont automatiquement mis à disposition :

- **config** : Map de propriétés de configuration
- **stepContext, stageContext, hookContext** : Contextes d'exécution de la step

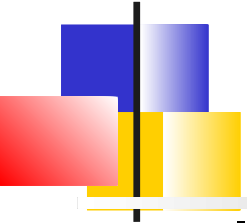
1. Attention, toutes les implémentations doivent avoir la même signature de méthode



Configuration de pipeline

La configuration de pipeline d'un projet consiste principalement à indiquer les librairies utilisées par le projet.

Cela peut se faire dans l'interface Jenkins ou via un fichier *pipeline_config.groovy*



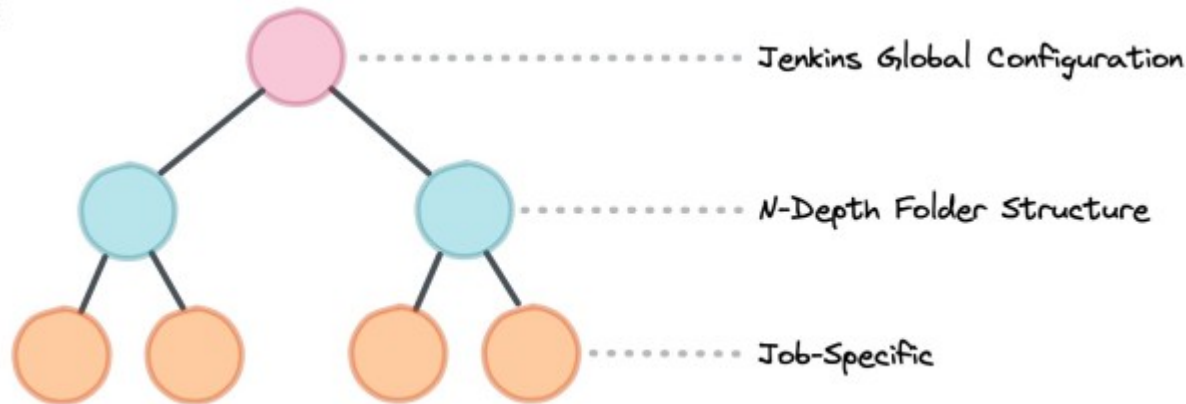
Catalogue de pipelines

Il est peu probable que toutes les pipelines d'une organisation correspondent à un seul modèle de pipeline.

JTE prend en charge plusieurs modèles de pipeline parmi lesquels les équipes peuvent choisir.

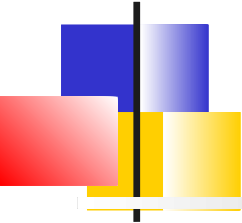
Ces modèles sont organisés en catalogues de pipelines ayant des relations hiérarchiques.

Governance tier



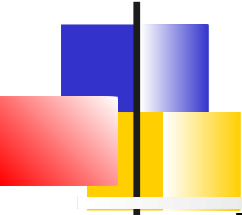
Les nœuds de la hiérarchie de configuration nommé Governance tiers stockent :

- Un ensemble de gabarits de pipeline
- Une configuration de pipeline héritée
- Les sources des librairies



Annexes

Tests et analyse statique
Infrastructure as Code
Le plugin JTE
Syntaxe script



Interpolation de String

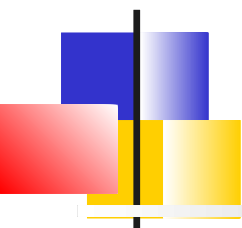
Jenkins a accès au mécanisme d'interpolation de chaîne de Groovy

Les chaînes de caractères peuvent être définies avec des simple ou double quotes :

```
def singlyQuoted = 'Hello'  
def doublyQuoted = "World"
```

Seules les doubles quotes supportent l'interpolation :

```
def username = 'Jenkins'  
echo "I said, Hello Mr. ${username}"
```



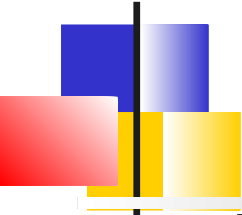
Variables d'environnement

Les variables d'environnement sont accessibles en lecture via :

```
// Declarative and Script //  
${env.BUILD_ID}
```

L'écriture d'une variable d'environnement est différent selon le type de syntaxe

- Directive environment (declaratif)
- Etape withEnv (script)

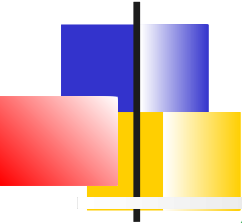


Paramètres

Les pipeline déclaratives supportent directement les paramètres fournis par les utilisateurs via la directive ***parameters***

Les pipelines scriptées implémentent les paramètres avec l'étape ***properties*** (voir Snippet Generator)

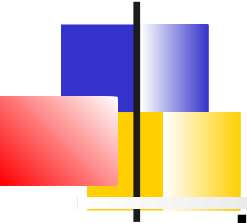
Si la pipeline a été configurée avec l'option Build with Parameters, les paramètres sont accessibles via la variable params



Exemple Paramètres

```
// Declarative //
pipeline {
  agent any
  parameters {
    string(name: 'Greeting', defaultValue: 'Hello', description: 'How should I greet
    the world?')
  }
  stages {
    stage('Example') {
      steps { echo "${params.Greeting} World!" }
    }
  }
}

// Script //
properties([parameters([string(defaultValue: 'Hello', description: 'How should I
  greet the world?', name: 'Greeting'))])])
node {
  echo "${params.Greeting} World!"
}
```



Gestion des erreurs

Les pipeline déclaratives peuvent traiter les erreurs via la section post qui permet de déclarer plusieurs "post conditions" comme :

- always, unstable, success, failure, et changed

Les pipelines scriptées reposent sur le mécanisme de Groovy

try/catch/finally